# DISEASE DIAGNOSIS SYSTEM

# AI DOC

A PROJECT REPORT

***Submitted by***

**AADHITHYAKRISHNAN K. H. (SNG22CS001)**

**ASWIN BABURAJ (SNG22CS052)**

**BENSON B. VARGHESE (SNG22CS057)**

**BHAVANA S. NAIR (SNG22CS058)**

**to**

**The APJ Abdul Kalam Technological University**

**in partial fulfillment of the requirements for the award of the Degree of**

***Bachelor of Technology***

***In***

***Computer Science and Engineering***


**Department of Computer Science and Engineering**

Sree Narayana Gurukulam College of Engineering,

Kadayiruppu, 682311

April 2025

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING,**
**SREE NARAYANA GURUKULAM COLLEGE OF ENGINEERING,**
**KADAYIRUPPU, 682311**

(Affiliated to APJ Abdul Kalam Technological University & Approved by A.I.C.T.E)

## CERTIFICATE

This is to certify that the project report, "**Disease Diagnosis System – AI Doc"** submitted by **Aadhithyakrishnan K. H. , Aswin Baburaj, Benson B. Varghese, Bhavana S. Nair** to the Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision .This report in any form has not been submitted to any other University or Institute for any purpose.

| **Prof. (Dr.) Smitha Suresh** | **Assoc. Prof. Sindhu M P** | **Assoc. Prof. Saini Jacob Soman** |
|---|---|---|
| **HEAD OF THE DEPARTMENT** | **PROJECT GUIDE** | **PROJECT COORDINATOR** |

Submitted for the University Evaluation on:

University Register Number :

Internal Examiner External Examiner

# DECLARATION

We undersigned hereby declare that the project report **"Disease Diagnosis System – AI Doc"**, submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under supervision of **Assoc. Prof. Sindhu M. V**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Kadayiruppu

Date : 03/04/2025

Aadhithyakrishnan K H
Aswin Baburaj
Benson B Varghese
Bhavana S Nair

# ACKNOWLEDGEMENT

Dedicating this project to the Almighty God whose abundant grace and mercy enabled its successful completion, we would like to express our profound gratitude to all the people who had inspired and motivated us to undertake this project.

We wish to express our sincere thanks to our Head of the Department, **Prof. (Dr.) Smitha Suresh**, for providing us the opportunity to undertake this project. We are deeply indebted to our project coordinator **Assoc. Prof**. **Saini Jacob Soman** and project guide **Assoc. Prof**. **Sindhu MP** in the Department of Computer Science and Engineering for providing us with valuable advice and guidance during the course of the project.

Finally, we would like to express my gratitude to Sree Narayana Gurukulam College of Engineering for providing me with all the required facilities without which the successful completion of the project work would not have been possible.

# COURSE OUTCOMES AND PROGRAM OUTCOMES

COURSE OUTCOMES: After the completion of the course the student will be able to

| | |
|---|---|
| CO1 | Identify technically and economically feasible problems (Cognitive Knowledge Level: Apply) |
| CO2 | Identify and survey the relevant literature for getting exposed to related solutions and get familiarized with software development processes (Cognitive Knowledge Level: Apply) |
| CO3 | Perform requirement analysis, identify design methodologies and develop adaptable & reusable solutions of minimal complexity by using modern tools & advanced programming techniques (Cognitive Knowledge Level: Apply) |
| CO4 | Prepare technical report and deliver presentation (Cognitive Knowledge Level: Apply) |
| CO5 | Apply engineering and management principles to achieve the goal of the project (Cognitive Knowledge Level: Apply) |

Engineering Graduates will be able to:

## Program outcomes:

| | |
|---|---|
| PO1 | Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems. |
| PO2 | Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences. |
| PO3 | Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations. |
| PO4 | Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions. |
| PO5 | Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations. |
| PO6 | The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice. |
| PO7 | Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development. |

| | |
|---|---|
| PO8 | Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. |
| PO9 | Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings. |
| PO10 | Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions. |
| PO11 | Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments. |
| PO12 | Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change |

| PSO | PROGRAM SPECIFIC OUTCOMES (PSO's) |
|---|---|
| PSO1 | Shall enhance the employability skills by finding innovative solutions for challenges and problems in various domains of CS. |
| PSO2 | Shall apply the acquired knowledge to develop software solutions and innovative mobile applications for various problems. |

## CO PO PSO MAPPING

| | PO1(Engineering Knowledge) | PO2(Problem Analysis) | PO3(Design/Development of solutions) | PO4(Investigation of complex problems) | PO5(Modern tool usage) | PO6(The Engineer and society) | PO7(Environment and Sustainability) | PO8(Ethics) | PO9(Individual and team work) | PO10(Communication) | PO11(Management and finance) | PO12(Life long learning) | PSO1(Finding innovative solutions) | PSO2(Software development) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| CO1 | 3 | 2 | 2 | 2 | | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| CO2 | 2 | 3 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 2 | 2 |
| CO3 | 2 | 2 | 3 | 3 | 3 | 2 | 2 | 2 | 3 | 3 | 2 | 2 | 3 | 3 |
| CO4 | 2 | 3 | 2 | 2 | 2 | 3 | | 2 | 2 | 2 | 2 | 2 | | 2 |
| CO5 | 2 | 2 | 3 | 3 | 3 | | 2 | 2 | 2 | | 3 | 2 | 3 | 3 |
| AVERAGE | 2.2 | 2.4 | 2.4 | 2.4 | 2.5 | 2.25 | 2 | 2 | 2.2 | 2.25 | 2.2 | 2.2 | 2.5 | 2.4 |

# PO PSO ATTAINMENT AND JUSTIFICATION

| PO | Attained point (0/1/2/3) | Justification |
|---|---|---|
| PO1 | 2 | Apply mathematical, scientific, and engineering principles, along with domain-specific knowledge, to develop advanced disease diagnosis systems. |
| PO2 | 2 | Identify and analyze complex medical and diagnostic problems using first principles of mathematics, data science, and biomedical engineering to derive meaningful insights. |
| PO3 | 3 | Design and develop innovative disease diagnosis models and systems that meet medical and healthcare needs while ensuring accuracy, reliability, and ethical considerations. |
| PO4 | 2 | Utilize research-based methods, conduct medical data analysis, apply machine learning techniques, and synthesize results to improve disease detection and treatment planning. |
| PO5 | 3 | Employ modern AI/ML techniques, data analytics, and healthcare technologies to enhance diagnostic accuracy while understanding their limitations. |
| PO6 | 2 | Assess the societal, legal, and ethical implications of disease diagnosis technologies, ensuring patient data privacy, safety, and adherence to medical regulations. |
| PO7 | 3 | Understand the impact of AI-driven diagnostic solutions on public health, resource optimization, and sustainable healthcare practices. |
| PO8 | 3 | Commit to professional ethics by ensuring transparency, fairness, and accountability in disease diagnosis models while addressing biases in medical data. |
| PO9 | 3 | Work effectively as an individual and collaborate with interdisciplinary teams, including medical professionals, data scientists, and engineers, to develop robust healthcare solutions. |
| PO10 | 3 | Effectively communicate technical and medical findings through reports, research papers, presentations, and discussions with medical practitioners and stakeholders. |
| PO11 | 3 | Demonstrate project management skills by efficiently handling resources, timelines, and budgets in the development of disease diagnosis systems, ensuring cost-effectiveness. |
| PO12 | 2 | Recognize the rapid advancements in healthcare technology and AI, staying updated with emerging trends in disease diagnosis for continuous improvement. |

| | | |
|---|---|---|
| PSO1 | 3 | We have utilized industry-standard software, frameworks, and libraries for designing and developing the disease diagnosis system. No trial-and-error approach was followed in software architecture design. Instead, a structured methodology was implemented using proper diagrams such as use case diagrams, ER diagrams, and system flowcharts to ensure clarity and precision. |
| PSO2 | 3 | Throughout the project, we explored various advanced technologies, methodologies, and concepts beyond our academic curriculum. Our primary focus was on fostering innovation in disease diagnosis. By studying research papers, blog posts, medical case studies, and relevant tutorials, we developed a problem-solving mindset and the ability to apply cutting-edge techniques to healthcare challenges. |

# ABSTRACT

The Disease Diagnosis Software is an innovative healthcare solution designed to provide users with intelligent, AI driven diagnostics for better health management. The platform incorporates advanced features such as women's period and ovulation tracking, mood detection, and an AI chatbot for real-time medical assistance. By leveraging cutting-edge technologies like machine learning and natural language processing, the software delivers personalized health insights, symptom analysis, and proactive alerts. This user-friendly tool ensures accessibility, data privacy, and comprehensive care, making it an indispensable companion for modern healthcare needs.

# TABLE OF CONTENTS

# LIST OF FIGURES AND TABLES

# LIST OF ABBREVIATIONS

| Abbreviations | Definition |
|---|---|
| AI | Artificial Intelligence |
| ML | Machine Learning |
| NLP | Natural Language Processing |
| HDNN | Hybrid Deep Neural Network |
| CNN | Convolutional Neural Network |
| LSTM | Long Short-Term Memory |
| DL | Deep Learning |
| ANN | Artificial Neural Network |
| SVM | Support Vector Machine |
| KNN | k-Nearest Neighbors |
| DT | Decision Tree |
| RF | Random Forest |
| AUC | Area Under the Curve |
| PO | Program Outcome |
| CO | Course Outcome |
| PSO | Program Specific Outcome |
| HIPAA | Health Insurance Portability and Accountability Act |
| GDPR | General Data Protection Regulation |
| AWS | Amazon Web Services |
| GCP | Google Cloud Platform |
| UI/UX | User Interface/User Experience |

| | |
|---|---|
| BVA | Boundary Value Analysis |
| EP | Equivalence Partitioning |
| WHO | World Health Organization |
| EHR | Electronic Health Records |
| IoT | Internet of Things |
| SMOTE | Synthetic Minority Over-sampling Technique |
| XG Boost | Extreme Gradient Boosting |
| ARIMA | Autoregressive Integrated Moving Average |
| GM (1,1) | Grey Model (First-order, one-variable) |
| IVIG | Intravenous Immunoglobulin |
| MDA | Microbe-Disease Association |
| miRNA | MicroRNA |
| BLS | Broad Learning System |
| RFE | Recursive Feature Elimination |
| OAuth 2.0 | Open Authorization 2.0 |
| MFA | Multi-Factor Authentication |
| SSL/TLS | Secure Sockets Layer/Transport Layer Security |
| RBAC | Role-Based Access Control |
| PCOS | Polycystic Ovary Syndrome |
| CDC | Centers for Disease Control and Prevention |
| PDF | Portable Document Format |
| CSV | Comma-Separated Values |
| API | Application Programming Interface |
| BP | Blood Pressure |
| ER | Entity-Relationship (Diagram) |

| | |
|---|---|
| Flutter | A UI toolkit by Google for building natively compiled apps |
| Dart | Programming language used for Flutter development |

# CHAPTER 1
# INTRODUCTION

## 1.1 Disease Diagnosis Software: An AI-Powered Healthcare Solution

The Disease Diagnosis Software is an innovative AI-driven healthcare platform designed to provide users with comprehensive health management tools. By harnessing the power of machine learning (ML) and natural language processing (NLP), this software delivers intelligent diagnostics, personalized health insights, and real-time medical assistance. Its primary goal is to enhance accessibility, accuracy, and efficiency in healthcare, enabling users to take proactive steps in monitoring their health.

## 1.2 Key Features

- AI-Powered Disease Diagnosis

  The platform employs advanced ML algorithms to analyse symptoms and generate preliminary disease predictions. By comparing user-inputted symptoms with vast medical datasets, the software assists in identifying potential health conditions, empowering users to make informed decisions regarding their well-being. This feature serves as a valuable tool for early disease detection and preventive care.

- Women's Health Monitoring

  Recognizing the importance of reproductive health, the software includes specialized features for women, such as tracking menstrual cycles, ovulation periods, and fertility windows. By offering predictive insights and reminders, it helps women manage their reproductive health with ease and accuracy.

- Mood Journal and Emotional Well-being

  Mental health is as crucial as physical health. The software allows users to record their daily moods and track emotional patterns over time. This feature enables individuals to identify triggers, assess mental health trends, and adopt effective self-care practices. Additionally, it provides recommendations for improving mental well-being.

- AI Chatbot for Medical Assistance

  The integrated AI chatbot enhances user engagement by providing instant responses to health-related queries. It suggests preventive measures, offers medical guidance based on

symptom analysis, and directs users to seek medical help when necessary. This chatbot ensures that users have access to immediate assistance, reducing anxiety and improving healthcare accessibility.

- Proactive Health Alerts

  By analysing user health data, the software generates real-time alerts and recommendations. These alerts notify users about potential health risks, suggest preventive measures, and remind them about necessary check-ups or vaccinations. This proactive approach facilitates early interventions, preventing the progression of illnesses.

- Personalized Insights and Reports

  Users receive tailored health reports based on their medical history, lifestyle choices, and symptom trends. These reports offer valuable insights, enabling individuals to make informed healthcare decisions. The software also allows users to share reports with healthcare providers, ensuring better diagnosis and treatment plans.

## 1.3 Technological Framework

The Disease Diagnosis Software is built using cutting-edge AI and cloud computing technologies, ensuring scalability, reliability, and efficiency. The ML models powering the platform are trained on extensive medical datasets, enhancing their diagnostic accuracy. NLP capabilities enable seamless human-computer interaction, allowing users to input symptoms in natural language.

To ensure user data privacy and security, the software implements advanced encryption techniques and adheres to global healthcare regulations such as HIPAA and GDPR. By maintaining compliance with stringent security standards, it safeguards sensitive health information, fostering trust among users.

## 1.4 Impact on Healthcare

This AI-powered solution bridges the gap between healthcare providers and patients by offering preliminary insights that reduce dependency on immediate clinical visits. By optimizing medical resources, it alleviates the burden on healthcare facilities and enables professionals to focus on critical cases.

Additionally, the software promotes early disease detection and preventive care, significantly improving health outcomes. By democratizing healthcare information, it empowers users to

take charge of their health, making diagnostics more accessible, convenient, and cost-effective.

# CHAPTER 2

## LITERATURE SURVEY

[1] Heart disease (HD) is the leading cause of global mortality, accounting for 17.9 million deaths annually. Early detection and accurate prediction are crucial for timely medical interventions and improved patient outcomes. This study proposes a robust HD prediction system using Hybrid Deep Neural Networks (HDNNs), combining Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), and Long Short-Term Memory (LSTM) to extract relevant features and capture complex patterns. The proposed system achieved a promising accuracy of 98.86% on two publicly available HD datasets, outperforming conventional systems. These results suggest that the proposed HDNN system has great potential to develop advanced and reliable HD prediction models, significantly contributing to medical diagnosis and improved patient care.

[2] This study proposes a Disease Prediction Model (DPM) to predict type 2 diabetes and hypertension based on individual risk factors. The DPM uses isolation forest to remove outlier data, synthetic minority oversampling technique to balance data, and an ensemble approach to predict diseases. The model achieved high accuracy and was used to develop a mobile application that collects risk factor data, sends it to a remote server for diagnosis, and provides immediate prediction results to help prevent and reduce health risks, enabling early intervention for type 2 diabetes and hypertension.

[3] Heart disease is a leading cause of mortality worldwide, and predicting cardiovascular disease is a significant challenge. Machine learning (ML) has shown promise in analyzing large healthcare datasets to make predictions. This study proposes a novel method using ML techniques to identify key features and improve prediction accuracy. A hybrid random forest with linear model (HRFLM) achieved an accuracy level of 88.7%, outperforming other classification techniques. This approach aims to enhance the prediction of cardiovascular disease, ultimately improving patient outcomes.

[4] This article compares three prediction models - ARIMA, grey model, and BP neural

network - for epidemic disease prediction, analyzing data from Shandong from 2014 to and 2019. The study finds that ARIMA is suitable for seasonal epidemic prediction in densely populated areas, while the grey model is ideal for short-term prediction with limited data, making it suitable for grassroots personnel. In contrast, the BP neural network offers high accuracy but has a complicated prediction process, making it more suitable for scientific research institutions.

[5] This study develops a multi-stage method to predict intravenous immunoglobulin resistance in Kawasaki disease patients, addressing issues with incomplete clinical data and machine learning model interpretability. The approach integrates co-clustering, group Lasso feature selection, and Explainable Boosting Machine, enabling group-based feature selection, missing data imputation, and patient subgroup-specific predictive models. Using real-world Electronic Health Records, the framework demonstrates superior performance compared to benchmark methods, highlighting the potential of data-driven approaches to identify high-risk individuals and inform healthcare decision-making.

[6] This study introduces MDADP, a webserver that identifies potential microbe-disease associations (MDAs). It combines a new MDA database with interactive prediction tools, integrating eight computational models to identify disease-related microbes. With 2019 known MDAs, MDADP provides interactive features and effective tools for researchers, making it a valuable resource for microbiology and disease-related research.

[7] This study proposes a computational approach, MISSIM, based on Broad Learning System (BLS) to predict disease-associated miRNAs. MISSIM introduces incremental learning to adapt to new data without forgetting prior knowledge. It achieves high performance (AUC 0.9400) and outperforms existing methods. Case studies on breast, lung, and esophageal neoplasms confirm the accuracy of MISSIM's predictions, demonstrating its potential value in biomedical research.

[8] Chronic diseases are a major global health concern, requiring early diagnosis and personalized treatment. This study proposes an advanced method to predict chronic diseases like heart attack, diabetes, breast cancer, and kidney disease at an early stage using a combination of powerful techniques. First, we tackle the complexity of large

medical datasets by applying Feature Engineering using Recursive Feature Elimination (RFE) with a Support Vector Machine (SVM). This step helps remove unnecessary features, making the data simpler and more efficient to work with. Next, the cleaned dataset is fed into the eXtreme Gradient Boosting (XGBoost) model—an effective machine learning algorithm known for accurately predicting complex patterns in data. To further boost performance, we use Bayesian optimization to fine-tune the model's settings. This method smartly explores different combinations of parameters to find the best ones for improving prediction accuracy. Overall, this approach significantly improves early detection of chronic diseases and highlights the strength of combining feature selection, ensemble learning, and optimization.

# CHAPTER 3

# PROBLEM STATEMENT AND OBJECTIVE

## 3.1 Problem Statement: Enhancing Disease Tracking with Machine Learning

Traditional disease tracking systems depend heavily on manual data collection and analysis, which poses significant challenges in effectively monitoring and responding to disease outbreaks. One of the primary issues with these systems is slow data processing. Since manual methods require substantial time for gathering, processing, and analyzing information, there is often a delay in identifying outbreaks and implementing timely responses, which can be critical in managing the spread of infectious diseases. Another concern is the error-prone nature of manual data handling. Human errors in data entry, reporting, and interpretation can result in inaccurate disease tracking, potentially leading to flawed decisions and compromised public health strategies. These inaccuracies hinder the ability of health authorities to respond effectively. Additionally, traditional systems suffer from limited scalability. As disease data increases—especially in densely populated areas or during large-scale outbreaks—manual systems struggle to efficiently manage and analyze this growing volume of information. This limitation affects the overall responsiveness and adaptability of the system. A further drawback is the lack of real-time insights. Manual processes rarely provide immediate alerts or updates, making it difficult for healthcare authorities to react quickly. The absence of real-time data significantly delays the containment efforts during emerging outbreaks, which is crucial for public safety. Lastly, traditional tracking approaches have inefficient predictive capabilities. Without the use of advanced analytics or machine learning, these systems are unable to accurately forecast disease trends or potential hotspots. This restricts proactive planning and reduces the effectiveness of preventive measures aimed at controlling the spread of diseases.

## 3.2 Proposed Solution: Machine Learning-Based Disease Tracking System

A machine learning-driven disease tracking system effectively addresses the limitations of traditional methods by introducing automation, real-time responsiveness, and predictive intelligence. By automating data collection and analysis, the system minimizes human error and accelerates the monitoring process, ensuring more accurate and timely insights. Real-time alerts enable healthcare authorities to respond promptly to emerging outbreaks, reducing the risk of widespread transmission. Additionally, the integration of predictive models, built on historical data, enhances the system's ability to forecast disease spread and support proactive decision-making. With its ability to handle vast datasets efficiently, the system offers excellent scalability, making it suitable for large-scale implementation. Overall, the use of machine learning significantly strengthens disease surveillance, early warning mechanisms, and outbreak prevention efforts, contributing to improved public health outcomes.

## 3.3 Objective: Developing an AI-Powered Disease Tracking and Prediction System

### 3.3.1 Primary Objective

The goal of this system is to accurately track and predict the spread of diseases by leveraging historical and real-time data. By utilizing machine learning algorithms, the system aims to enhance public health responses through timely interventions, reducing the impact of outbreaks.

### 3.3.2 Key Focus Areas

A machine learning-based disease tracking system offers a comprehensive solution for modern public health challenges through several key capabilities. It enables real-time disease monitoring by collecting and processing data from diverse sources such as hospitals, public health records, and online reports, ensuring continuous surveillance to detect patterns and anomalies at an early stage. Leveraging predictive analytics, the system uses machine learning models to forecast potential outbreaks based on historical data, helping identify future hotspots and allowing for the implementation of timely preventive measures. Through automated alerts and early warning mechanisms, the

system notifies healthcare authorities and policymakers when unusual trends are detected, significantly improving response times and enabling effective containment strategies. Additionally, it supports data-driven decision making by offering visualized reports and interactive dashboards, making it easier for healthcare professionals, researchers, and policymakers to interpret trends and make informed decisions. With its high scalability and adaptability, the system is capable of handling vast datasets across global, national, and regional levels. It also evolves with emerging health crises and new disease strains by continuously learning from new data, making it a powerful and flexible tool for public health management.

## 3.4 Expected Impact

The implementation of a machine learning-driven disease tracking system brings several significant benefits to public health management. One of the most crucial advantages is the early detection of outbreaks, which allows for quicker interventions and reduces the impact of disease spread. By predicting case surges, the system also enables improved resource allocation for healthcare facilities, ensuring that critical supplies and personnel are directed where they are most needed. This proactive approach contributes to reduced transmission rates, as timely containment strategies can be implemented before the situation escalates. Moreover, the insights provided by such a system support better public health planning, equipping authorities with the data and foresight necessary to prepare for and manage future pandemics and epidemics more effectively.

# CHAPTER 4

# REQUIREMENT SPECIFICATION

## 4.1 Functional Requirements

The system is designed to offer core functionalities aimed at diagnosing diseases, tracking health metrics, and empowering users in maintaining their overall well-being. By incorporating cutting-edge AI technologies, it ensures precise and intelligent health monitoring, tailored for each individual's unique profile.

### 4.1.1 User Management

The platform offers robust user management with a seamless registration and authentication process that supports multiple login options including Email, Phone, Google, and Social Login. Security is prioritized through OAuth 2.0 and multi-factor authentication (MFA), along with user-friendly password recovery options. Users can create detailed profiles by inputting personal information such as age, gender, medical history, and lifestyle habits. These profiles are customizable and benefit from AI-driven insights to enhance personalization. Role-Based Access Control (RBAC) ensures structured access where Admins oversee the system, Doctors analyze and consult based on patient data, Patients manage their health records, and Researchers access anonymized datasets for scientific progress.

### 4.1.2 Women’s Health Tracking

Specifically tailored for women, the system enables advanced menstrual and ovulation tracking using AI to forecast cycles with remarkable accuracy. A sleek and intuitive interface allows users to log symptoms and period dates effortlessly. Alerts notify users of health irregularities such as missed periods or signs of PCOS, while AI offers personalized lifestyle and dietary recommendations to support hormonal balance and overall reproductive health.

### 4.1.3 Mood Journal & Mental Health Monitoring

Users can log daily mood entries along with descriptions to better understand their emotional state. The system processes this data to present graphical trends that help identify patterns and emotional triggers. AI-generated insights recommend wellness strategies and encourage positive mental health practices. With regular reminders and gentle check-ins, the system promotes mindfulness and continuous self-reflection.

### 4.1.4 AI Chatbot for Medical Assistance

An intelligent AI-powered chatbot is available 24/7 to assist users with their health concerns. Utilizing natural language processing (NLP) and referencing medical authorities like WHO and Mayo Clinic, it delivers accurate health information. The integrated symptom checker evaluates real-time inputs and suggests whether medical attention is required. For critical cases, users can opt for virtual consultations through chat or video, ensuring prompt access to professional care.

### 4.1.5 Symptom Analysis & Disease Prediction

The system analyzes user-submitted symptoms in conjunction with extensive medical databases, offering instant diagnostic feedback. Sophisticated machine learning models evaluate inputs against medical history and lifestyle to predict possible conditions, aiding in early detection. A comprehensive risk factor analysis flags vulnerabilities to chronic illnesses, while early warning alerts keep users informed of potential health issues before they escalate.

### 4.1.6 Personalized Health Insights & Alerts

AI-generated health summaries help users visualize trends in their symptoms and overall wellness. The system ensures timely medication intake with intelligent reminders and integrates with wearables like Fitbit, Apple Watch, and Google Fit. Through this integration, it delivers real-time insights on heart rate, sleep, and activity levels—offering a truly holistic view of health.

### 4.1.7 Data Visualization & Reports

A dynamic dashboard presents health insights in an engaging and digestible format. Users can view visually appealing charts for tracking menstrual cycles, mood swings, and

symptom history. Reports can be easily exported in PDF or CSV formats for sharing with healthcare providers, and secure sharing options enhance the communication between patients and doctors.

### 4.1.8 Multi-Device & Multi-Language Support

Designed for versatility, the system is available on both web and mobile platforms, compatible with iOS and Android. It caters to a global audience with support for multiple languages and voice-based interactions bridging language barriers and enhancing user engagement across regions.

## 4.2 Non-Functional Requirements

### 4.2.1 Performance

Performance is key to user satisfaction, and the system delivers low-latency responses, particularly in chatbot interactions, for a seamless user experience. It is engineered to efficiently process vast medical datasets and support thousands of users simultaneously without lag.

### 4.2.2 Scalability

Built on a robust cloud-based architecture, the platform scales effortlessly to accommodate user growth. Microservices ensure modular, flexible expansion while distributing loads efficiently to maintain uninterrupted performance during peak times.

### 4.2.3 Security & Compliance

Data security is a top priority. The system leverages AES-256 encryption and SSL/TLS protocols to safeguard user information. It is fully compliant with HIPAA, GDPR, and other regional healthcare regulations. With RBAC, sensitive medical data is securely managed and accessed only by authorized personnel.

### 4.2.4 Reliability & Availability

A high-availability infrastructure guarantees 99.9% uptime, thanks to reliable cloud services such as AWS or GCP. Backup mechanisms and disaster recovery plans are in place to protect data integrity and ensure business continuity during unexpected events.

### 4.2.5 Usability & Accessibility

The user interface is intuitively designed for ease of use across all age groups. It supports both voice and text interaction for greater flexibility and accessibility. Additionally, the system is optimized for screen readers and assistive technologies, making it inclusive for users with disabilities.

## 4.3. System Architecture

### 4.3.1 Technology Stack

The frontend is developed using Flutter (Dart), providing a smooth, responsive experience across platforms. The backend harnesses the capabilities of Google Dialogflow for powerful AI chatbot functions and natural language processing. Data storage is handled securely in the cloud, integrated seamlessly with Dialogflow. APIs manage secure user authentication and data exchange, ensuring a streamlined, efficient system that is both scalable and secure.

# CHAPTER 5

# SYSTEM DESIGN

## 5.1 SYSTEM ARCHITECTURE

This diagram illustrates a Realtime Conversation AI-Chatbot designed for personal healthcare support. The chatbot interacts with users to provide four key services: symptom prediction, real-time remedies, daily health tracking, and period tracking. By analyzing user input, it can predict possible health conditions, suggest immediate remedies, log daily health activities, and monitor menstrual cycles. The integration of these features allows for personalized and continuous health monitoring through natural, conversational interactions.

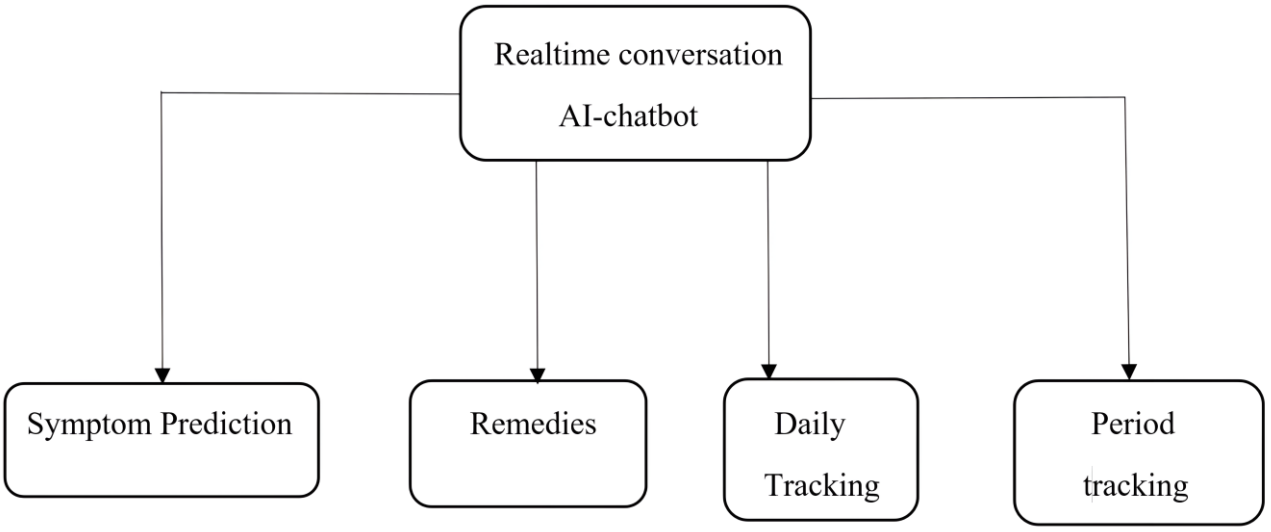


**Fig. 5.1: AI Chatbot**

This diagram outlines the structure of a Daily Tracking system focused on personal health management. It includes three key components: Fitness Check for monitoring physical activity and overall wellness, Acute Disease Check for identifying short-term illnesses or symptoms, and Chronic Disease Monitoring for managing long-term conditions like diabetes or hypertension. Together, these components enable comprehensive daily health supervision, helping users stay proactive about their fitness and medical conditions.

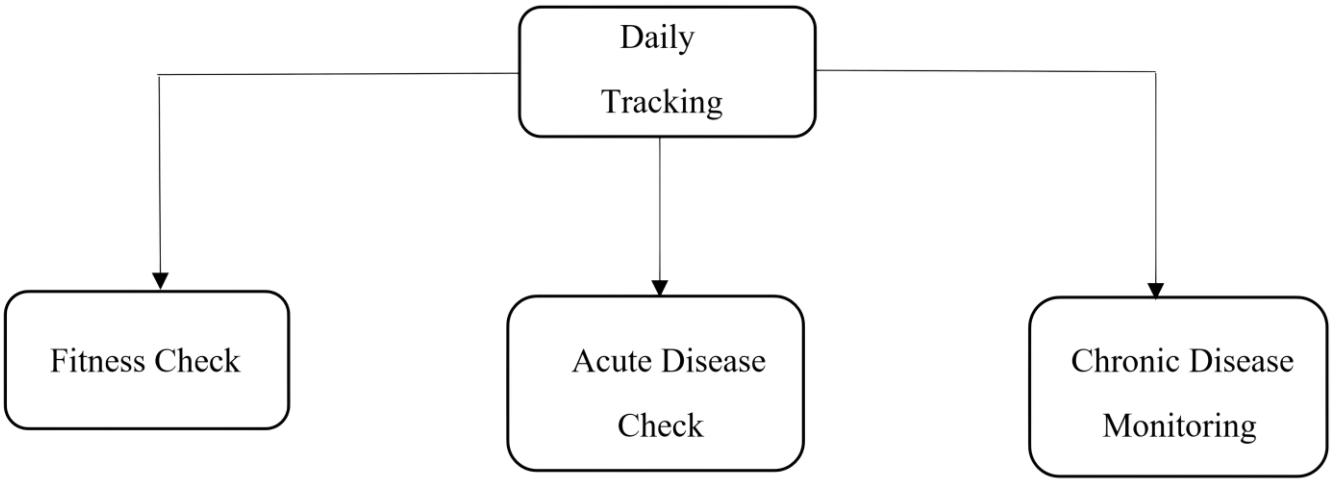


**Fig. 5.2: Daily Tracking**

This diagram represents a Personalized Health Tracking system, which focuses on individualized monitoring of key health metrics. It includes three main components: Daily BP and sugar check for keeping track of blood pressure and glucose levels, Personalized Goal for setting and achieving specific health objectives based on individual needs, and Exercise Check for monitoring physical activity levels. Together, these elements help users maintain a tailored and holistic approach to managing their daily health and wellness.

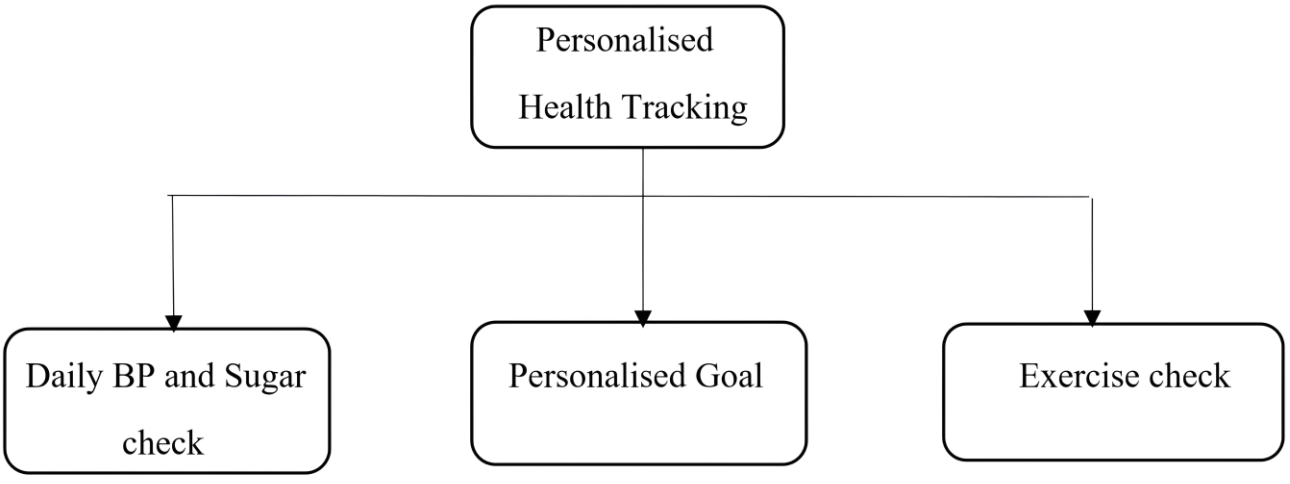


**Fig. 5.3: Health Tracking**

This diagram represents the components of a Period Tracking system designed to support users in managing their menstrual health more effectively. At the core is the period tracking module, which connects to five key features: Daily BP and sugar check for monitoring vital health indicators, Status for providing real-time menstrual health updates, Personalized Goal for setting individual health and wellness targets, Activities for suggesting or tracking physical and wellness routines, and Notification to remind users about upcoming cycles, health checks, or goals. Together, these features enable a holistic and personalized approach to period and overall health management.

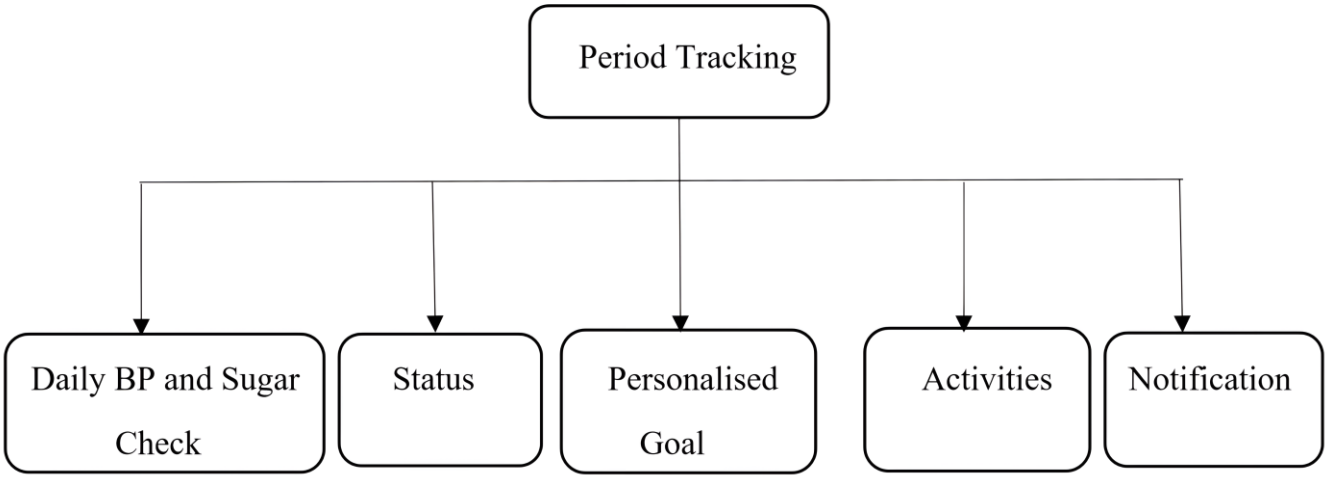


**Fig. 5.4: Period Tracking**

## 5.2 USE CASE DIAGRAM

This use case diagram outlines the interaction between different user roles and features in a health tracking system. There are three types of users: Free User, Registered User (divided into Male and Female), and Admin. Free users have access to basic features like Search, Symptom Prediction, and Remedies. Registered users can access more personalized services including Period Tracking, Daily Tracking, Real-Time Conversation, Mood Journal, Personalized Health Tracking, and Global Updates. Female users specifically have access to Period Tracking, while both male and female users share access to most other features. Admin users manage system operations through Analysis and Sorting functions, using data from all modules. This structure ensures tailored health support while maintaining organized backend control.

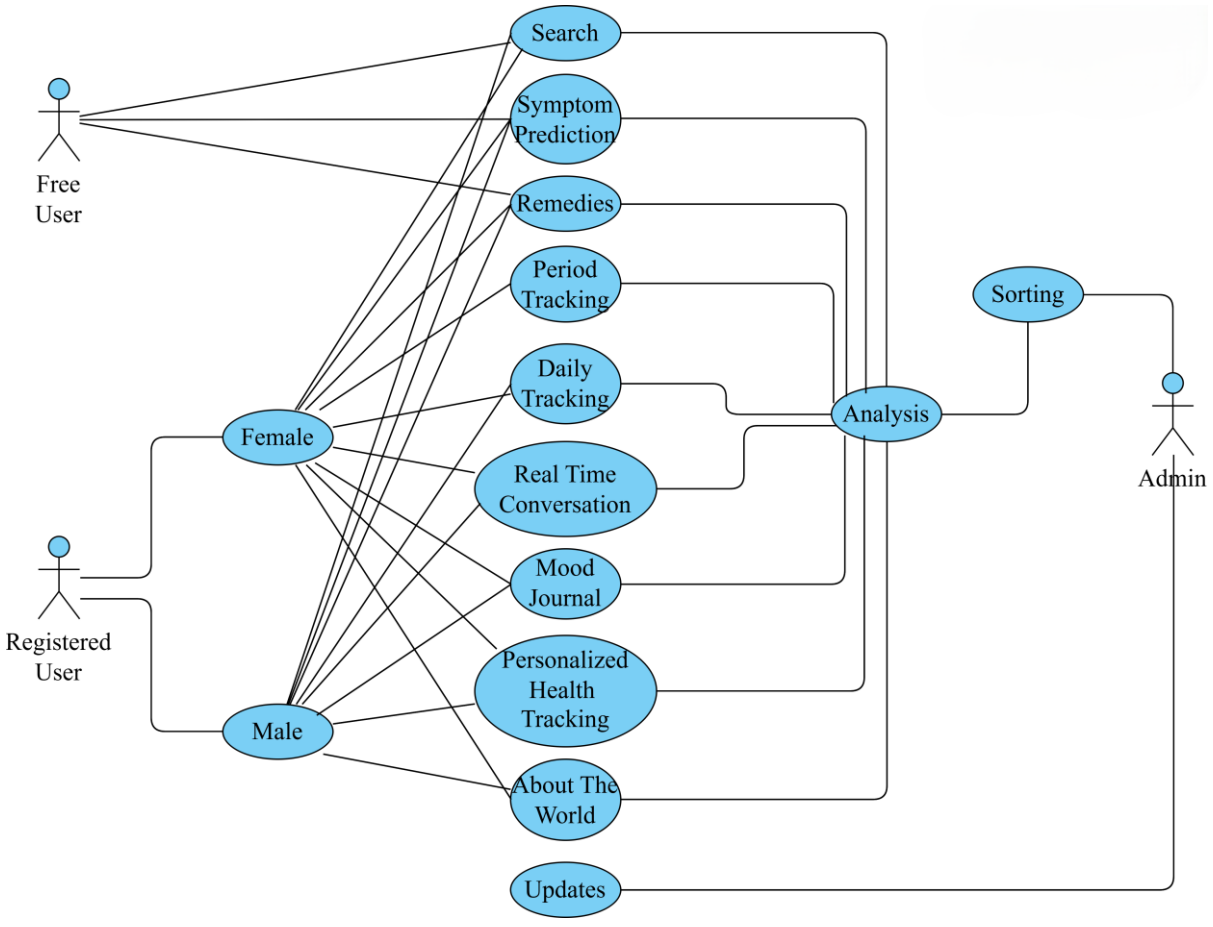


**Fig. 5.5: Use Case Diagram**

## 5.3 USER AND DATA PROCESSING FLOWCHART

This flowchart represents a comprehensive health monitoring system driven by user input. The system collects various data types: period and ovulation data, mood data, symptom reports, and medical queries. Each data type is preprocessed and then analyzed by respective modules—cycle prediction, mood detection, symptom analysis, and an AI chatbot. All outputs are funneled through a layer that ensures data security and privacy. Based on the analysis, the system provides actionable outcomes such as next period prediction, mood insights, chat-based health recommendations, disease predictions, and alerts for critical conditions, offering users a holistic and secure digital health assistant.

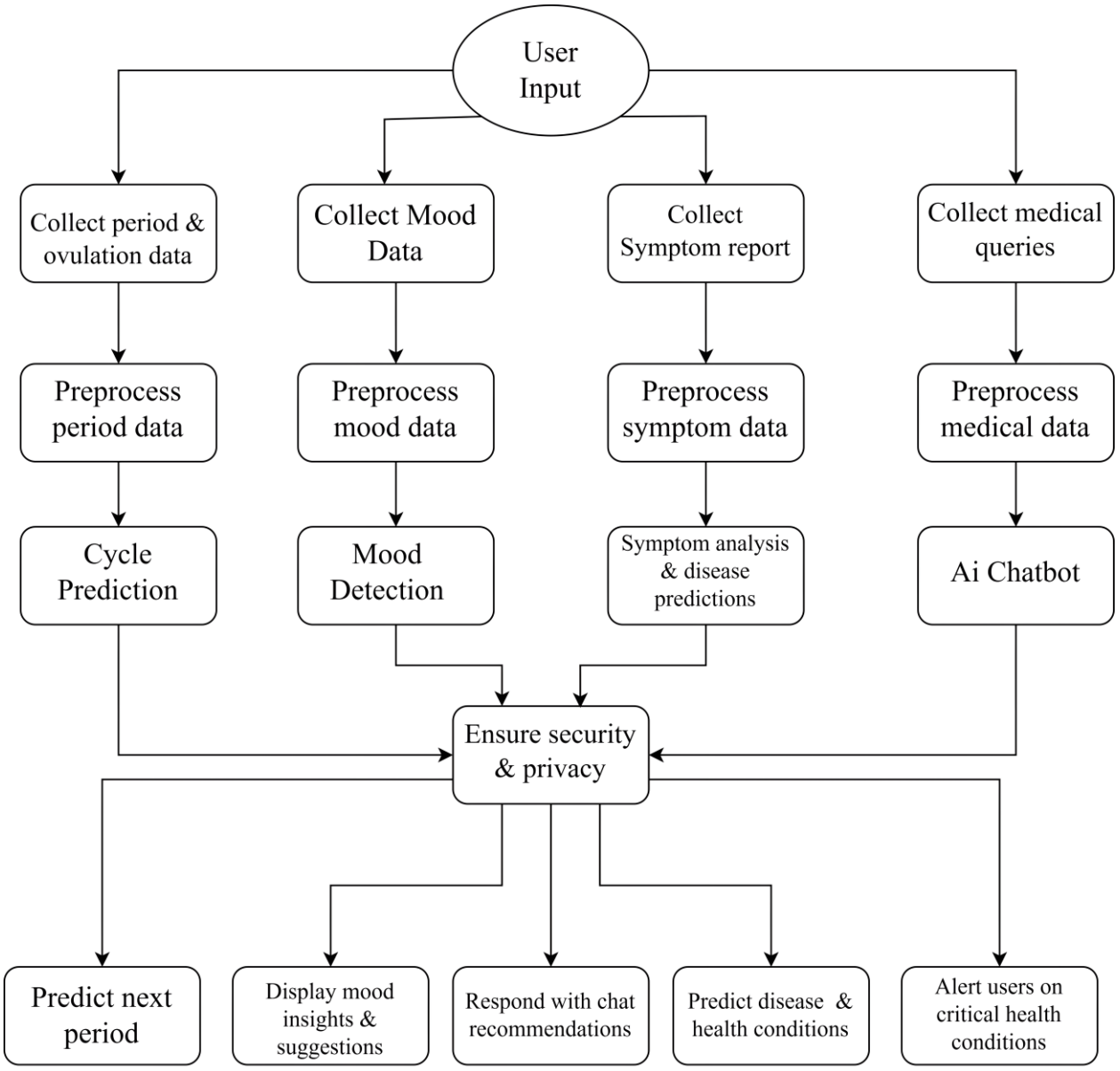


**Fig. 5.6: User and Data Processing Flowchart**

# CHAPTER 6

# SYSTEM IMPLEMENTATION

## PROGRAM CODES

### 6.1 Main.dart

```
import 'package:flutter/material.dart';
import 'package:table_calendar/table_calendar.dart';
import 'package:dialog_flowtter/dialog_flowtter.dart';
import 'package:image_picker/image_picker.dart';
import 'package:provider/provider.dart';
import 'package:url_launcher/url_launcher.dart';
import 'dart:io';
import 'dart:async';
import 'package:flutter_inappwebview/flutter_inappwebview.dart';
class AuthProvider with ChangeNotifier {
  bool _isLoggedIn = false;
  bool get isLoggedIn => _isLoggedIn;
   void login() {
    _isLoggedIn = true;
  notifyListeners();    }
  void logout() {
    _isLoggedIn = false;
    notifyListeners();  }}
void main() {
WidgetsFlutterBinding.ensureInitialized();
runApp(
    MultiProvider
     providers: [
       ChangeNotifierProvider(create: (_) => ThemeProvider()),
```

```
        ChangeNotifierProvider(create: (_) => UserProfile()),
        ChangeNotifierProvider(create: (_) => AuthProvider()),
      ],
      child: MyApp(),
    ),
  );
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return    Consumer<ThemeProvider>(
    builder: (context, themeProvider, _) {
       return MaterialApp(
         title: 'Medbot',
         debugShowCheckedModeBanner: false,
         theme: ThemeData.light().copyWith(
         primaryColor: Colors.deepPurple,
         colorScheme: ColorScheme.fromSwatch(
            primarySwatch: Colors.deepPurple,),
         scaffoldBackgroundColor: Colors.white, ),
         darkTheme: ThemeData.dark().copyWith(
         colorScheme: ColorScheme.dark(
            primary: Colors.deepPurple,
            secondary: Colors.purpleAccent),
          scaffoldBackgroundColor: Colors.grey[900] ),
         themeMode: themeProvider.themeMode,
         initialRoute: '/login',
         routes: {
           '/login': (context) => LoginPage(),
           '/signup': (context) => SignupPage(),
           '/home': (context) => HomePage(), },);};); }}
```

```
    required this.content,
    required this.type,
    required this.imageUrl });}
class HomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Medbot'),
        actions: [
        IconButton(
            icon: Icon(Icons.notifications),
            onPressed: () {}, ) ], ),
      drawer: AppDrawer(),
      body: SingleChildScrollView(
        child: Padding(
          padding: EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              _buildWelcomeHeader(context),
              SizedBox(height: 30),
              _buildHealthGrid(context),
              SizedBox(height: 20),
              _buildHealthTipsSection(), ],),),),
        floatingActionButton: FloatingActionButton(
        onPressed: () => Navigator.push(context,
            MaterialPageRoute(builder: (context) => ChatBotPage())),
        child: Icon(Icons.chat, color: Colors.white),
        backgroundColor: Theme.of(context).colorScheme.primary),);}
  Widget _buildWelcomeHeader(BuildContext context) {
    final user = Provider.of<UserProfile>(context);
```

```
return Container(
  padding:  EdgeInsets.all(20),
  decoration:  BoxDecoration(
  gradient: LinearGradient(
     colors: [
     Theme.of(context).colorScheme.primary,
     Theme.of(context).colorScheme.secondary,],
     begin: Alignment.topLeft,
     end: Alignment.bottomRight,),
  borderRadius: BorderRadius.circular(15),),
  child: Row(
   children: [
     Expanded(
     child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text("Welcome ${user.name}!",
             style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
                color: Colors.white)),
          SizedBox(height: 5),
          Text("Your health companion for better living",
             style: TextStyle(color: Colors.white70)),],),),
     GestureDetector(
          onTap: () => Navigator.push(context,
          MaterialPageRoute(builder: (context) => ProfilePage())),
       child: CircleAvatar(
          radius: 30,
          backgroundImage: user.profileImage != null
             ? FileImage(user.profileImage!)
             : null,
```

```
            child: user.profileImage == null
                ? Icon(Icons.person, size: 30, color: Colors.white)
                : null,),)],),);}
Widget _buildHealthGrid(BuildContext context) {
  List<Map<String, dynamic>> features = [
    {"title": "Symptom Checker",
     "icon": Icons.health_and_safety,
     "color": Colors.orange,
     "page": DiseaseDiagnosisPage()},
    {"title": "Cycle Tracker",
     "icon": Icons.female,
     "color": Colors.pink,
     "page": WomenHealthPage()},
    {"title": "Mood Journal",
     "icon": Icons.emoji_emotions,
     "color": Colors.blue,
     "page": MoodDetectionPage()},
    {"title": "Medication",
     "icon": Icons.medical_services,
     "color": Colors.green,
     "page": ChatBotPage()},
    {"title": "Emergency\nContacts",
     "icon": Icons.emergency,
     "color": Colors.red,
     "page": EmergencyContactsPage()},
    {"title": "Medical\nRecords",
     "icon": Icons.assignment,
     "color": Colors.teal,
     "page": MedicalRecordsPage()},];
    return GridView.builder(
    shrinkWrap: true,
    physics: NeverScrollableScrollPhysics(),
```

```
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 3, // 3 items per row
    crossAxisSpacing: 8, // Spacing between columns
  mainAxisSpacing: 8, // Spacing between rows
  childAspectRatio: 2.0, // Adjusted to make the cards square ),
  itemCount: features.length,
  itemBuilder: (context, index) => _buildFeatureCard(features[index], context), );}
Widget _buildFeatureCard(Map<String, dynamic> feature, BuildContext context) {
return Card(elevation: 2, // Reduced elevation for a flatter look
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(12), ),
  child: InkWell(borderRadius: BorderRadius.circular(12),
    onTap: () => Navigator.push(
      context, MaterialPageRoute(builder: (context) => feature["page"])),
    child: Container(
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(12),
        gradient: LinearGradient(
          colors: [feature["color"].withOpacity(0.1), Colors.white],
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,),),
      child: Padding(padding: EdgeInsets.all(8), // Adjusted padding to match the screenshot
        child: Column(mainAxisAlignment: MainAxisAlignment.center, // Centered content
          children: [Icon(feature["icon"],size: 30, // Keep the icon size the samecolor:
feature["color"],),
            SizedBox(height: 8), // Adjusted spacing between icon and text
            Text(feature["title"],
              style: TextStyle(
                fontSize: 13, // Keep the font size the same
                fontWeight: FontWeight.bold,
                color: Colors.deepPurple,),
              textAlign: TextAlign.center, // Centered text), ],  ),),),),); }
```

```
Widget _buildHealthTipsSection() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Padding(
        padding: EdgeInsets.symmetric(vertical: 10),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text(
              'Health Tips & News',
              style: TextStyle(
              fontSize: 20,
                fontWeight: FontWeight.bold,
                color: Colors.deepPurple,),),
            IconButton(
              icon: Icon(Icons.more_horiz, color: Colors.deepPurple),
              onPressed: ()
      HealthTipsCarousel(),],);}
    class AppDrawer extends StatelessWidget {
      @override
      Widget build(BuildContext context) {
        final user = Provider.of<UserProfile>(context);

        return Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: [
          UserAccountsDrawerHeader(
accountName: Text(user.name!), accountEmail: Text(user.email!),
```

```
  currentAccountPicture: CircleAvatar(
  backgroundImage: user.profileImage != null
      ? FileImage(user.profileImage!)
      : null,
    child: user.profileImage == null
      ? Icon(Icons.person, size: 40, color: Colors.white)
      : null,),
  decoration: BoxDecoration(
    color: Theme.of(context).colorScheme.primary,),),
FeatureItem(" Disease Diagnosis", Icons.local_hospital,
    DiseaseDiagnosisPage(), context),
FeatureItem("  Women's Health Tracking", Icons.female, WomenHealthPage(),
    context),
FeatureItem(" Mood Detection", Icons.mood, MoodDetectionPage(), context),
FeatureItem(" AI Medical Chatbot", Icons.smart_toy, ChatBotPage(), context),
Divider(),
ListTile(

  leading: Icon(Icons.settings),
  title: Text('Settings'),
  onTap: () => Navigator.push(context,
      MaterialPageRoute(builder: (context) => SettingsPage())),),
ListTile(
  leading: Icon(Icons.logout),
  title: Text('Logout'),


  onTap: () {
    context.read<AuthProvider>().logout();
    Navigator.pushReplacementNamed(context, '/login');
```

```
Widget FeatureItem(String title, IconData icon, Widget page, BuildContext context) {
  return ListTile(
    leading: Icon(icon, color: Theme.of(context).colorScheme.onSurface),
    title: Text(title),
    onTap: () {
      Navigator.pop(context);
      Navigator.push(context, MaterialPageRoute(builder: (context) => page));

class WomenHealthPage extends StatefulWidget {
  @override
  _WomenHealthPageState createState() => _WomenHealthPageState();}
class _WomenHealthPageState extends State<WomenHealthPage> {
CalendarFormat _calendarFormat = CalendarFormat.month;
  DateTime _focusedDay = DateTime.now();
  DateTime? _selectedDay;
  DateTime? _lastPeriodStart;
  int _cycleLength = 28;
  Map<DateTime, Map<String, dynamic>> _dailyData = {};
  // Maps each date to its predicted phase (e.g. "🔴 Period Day")
  Map<DateTime, String> _cyclePredictions = {};
  // List of predicted period start dates (first day of each cycle)
  List<DateTime> _predictedPeriodStarts = [];
  // Phase data: color and description.
  final Map<String, Map<String, dynamic>> _phaseData = {
    "🔴 Period Day": {
      "color": Colors.red,
      "description": "Menstrual phase: Shedding of the uterine lining" },
    "🟢 Follicular": {
      "color": Colors.green,
      "description": "Preparation for ovulation, estrogen levels rise"},
    "🟡 Ovulation": {"color": Colors.amber,
      "description": "Egg is released; highest chance of pregnancy" },
```

```
  "🟣 Luteal": { "color": Colors.purple,
    "description": "Body prepares for pregnancy or next cycle"}; };
// Helper function to compare dates (ignoring time)
 bool isSameDate(DateTime a, DateTime b) {
  return a.year == b.year && a.month == b.month && a.day == b.day; }
 // Calculate predictions for 6 cycles (roughly 5-6 months) based on the given start date.
 void _calculatePredictions(DateTime start) {
  _cyclePredictions.clear();
  _predictedPeriodStarts.clear();
  final int menstrualDays = 5;
  final int follicularDays = _cycleLength - 14 - menstrualDays;
  for (int i = 0; i < 6; i++) {
   final DateTime cycleStart = start.add(Duration(days: i * _cycleLength));
   _predictedPeriodStarts.add(cycleStart);
   // Menstrual phase
   for (int j = 0; j < menstrualDays; j++) {
    _cyclePredictions[cycleStart.add(Duration(days: j))] = "🔴 Period Day";}
   // Follicular phase
   for (int j = menstrualDays; j < menstrualDays + follicularDays; j++) {
    _cyclePredictions[cycleStart.add(Duration(days: j))] = "🟢 Follicular";}
   // Ovulation day
   _cyclePredictions[cycleStart.add(Duration(days: menstrualDays + follicularDays))] =
      🥱 Ovulation";
   // Luteal phase
   for (int j = menstrualDays + follicularDays + 1; j < _cycleLength; j++) {
    _cyclePredictions[cycleStart.add(Duration(days: j))] = "🟣 Luteal";}}
  setState(() {});}
 // Let the user update cycle length.
 void _showCycleSettings() async {
 final result = await showDialog<int>(
   context: context,
   builder: (context) => AlertDialog(
```

```
      title: Text("Cycle Settings"),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text("Average cycle length (days):"),

          SizedBox(height: 10),
          TextField(
            keyboardType: TextInputType.number,
            decoration: InputDecoration(
              border: OutlineInputBorder(),
              hintText: '$_cycleLength',),
            onChanged: (value) => _cycleLength = int.tryParse(value) ?? 28,),],),
      actions: [TextButton(
          onPressed: () => Navigator.pop(context, _cycleLength),
          child: Text("Save"),),],),);
  if (result != null) {
    setState(() => _cycleLength = result);
    if (_lastPeriodStart != null) _calculatePredictions(_lastPeriodStart!); }}
// Log daily data (symptoms, flow, and notes).
void _logDailyData(DateTime day) async {
  final result = await showDialog<Map<String, dynamic>>(
    context: context,
    builder: (context) => DailyLogDialog(initialData: _dailyData[day]),);
  if (result != null) {
    setState(() => _dailyData[day] = result); }}
// When a day is tapped, show a dialog with two actions:
// 1. Log Daily Data, 2. Set as Period Start.
void _onDayTapped(DateTime selectedDay, DateTime focusedDay) {
  setState(() {
    _selectedDay = selectedDay;
    _focusedDay = focusedDay;});
  showDialog(
```

```
      context: context,
      builder: (context) => AlertDialog(
        title: Text("Select Action"),
        content: Text(
            "What would you like to do on ${selectedDay.toLocal().toString().substring(0,
10)}?"),
        actions: [
          TextButton(
          onPressed: () {
              Navigator.pop(context);
              _logDailyData(selectedDay); },
          child: Text("Log Daily Data"),),
          TextButton(
            onPressed: () {
              setState(() {
                _lastPeriodStart = selectedDay;
                _calculatePredictions(selectedDay); });
              Navigator.pop(context); },
            child: Text("Set as Period Start"),),
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: Text("Cancel")
  // Build a card showing cycle info: cycle length and next predicted period date.
  Widget _buildCycleInfoCard() {
    String predictedNextPeriod = "-";
    if (_predictedPeriodStarts.isNotEmpty) {
      DateTime next = _predictedPeriodStarts.firstWhere(
          (date) => date.isAfter(DateTime.now()),
          orElse: () => _predictedPeriodStarts.last);
    predictedNextPeriod = next.toString().substring(0, 10); }
    return Card(
      elevation: 4,
      color: Colors.deepPurple.shade50,
```

```
    child: Padding(
      padding: EdgeInsets.all(16),
      child: Column(
        children: [
          Text("Cycle Information",
              style: TextStyle(
              fontSize: 18,
                fontWeight: FontWeight.bold,
                color: Colors.deepPurple,)),
          SizedBox(height: 10),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceAround,
            children: [
              _buildInfoItem("Cycle Length", "$_cycleLength days"),
              _buildInfoItem("Next Period", predictedNextPeriod),],),],), ); }
Widget _buildInfoItem(String title, String value) {
  return Column(
    children: [
      Text(title, style: TextStyle(color: Colors.grey.shade600)),
      Text(value,
          style: TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.bold,
            color: Colors.deepPurple,)),],); }
// Build the calendar with the custom on-tap action.
Widget _buildCalendar() {
  return Card(
    elevation: 4,
    child: Padding(
      padding: EdgeInsets.all(12),
      child: TableCalendar(
        firstDay: DateTime.now().subtract(Duration(days: 365)),
```

```
lastDay: DateTime.now().add(Duration(days: 365)),
focusedDay: _focusedDay,
calendarFormat: _calendarFormat,
  headerStyle: HeaderStyle(
  titleCentered: true,
  formatButtonVisible: false,
  headerPadding: EdgeInsets.symmetric(vertical: 8),
  titleTextStyle: TextStyle(
    color: Colors.deepPurple,
    fontWeight: FontWeight.bold,),
  leftChevronIcon: Icon(Icons.chevron_left, color: Colors.deepPurple),
rightChevronIcon: Icon(Icons.chevron_right, color: Colors.deepPurple),),
calendarStyle: CalendarStyle(
  defaultTextStyle: TextStyle(color: Colors.black87),
  weekendTextStyle: TextStyle(color: Colors.black87),
  outsideTextStyle: TextStyle(color: Colors.grey),
  selectedDecoration: BoxDecoration(
    color: Colors.deepPurple.shade100,
  shape: BoxShape.circle,),
  todayDecoration: BoxDecoration(
  color: Colors.deepPurple.shade50,
  shape: BoxShape.circle,),),
selectedDayPredicate: (day) => isSameDate(_selectedDay ?? DateTime.now(), day),
onDaySelected: _onDayTapped,
calendarBuilders: CalendarBuilders(
  defaultBuilder: (context, day, focusedDay) {
  final String? phase = _cyclePredictions[day];
  final data = _dailyData[day];
    return Container(
      margin: EdgeInsets.all(4),
      decoration: BoxDecoration(
      color: phase != null
          ? _phaseData[phase]!["color"].withOpacity(0.2)
```

```
                : null,
            shape: BoxShape.circle,
              border: data != null ? Border.all(color: Colors.deepPurple) : null,  ),
          child: Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Text(
                  "${day.day}",
                  style: TextStyle(
                    color: phase != null
                        ? _phaseData[phase]!["color"]
                        : Colors.black87,
                    fontWeight: FontWeight.bold,),),
                if (phase != null)
                  Text(
                    phase,
                    style: TextStyle(
                        fontSize: 8, color: _phaseData[phase]!["color"]),
                    textAlign: TextAlign.center,),
                if (data?['flow'] != null)
                  Icon(Icons.water_drop,
                      color: _getFlowColor(data!['flow']), size: 12),],),
// Build the legend for cycle phases.
Widget _buildPhaseLegend() {
return Wrap(
    spacing: 10,
    runSpacing: 10,
    alignment: WrapAlignment.center,
    children: _phaseData.entries
        .map((entry) => Chip(
            backgroundColor: entry.value["color"].withOpacity(0.2),
            label: Text(entry.key, style: TextStyle(color: Colors.black87)),
```

```
            avatar: Icon(Icons.circle, color: entry.value["color"], size: 18),
                ))
        .toList(),
   // Build the daily log section if log data exists for the selected day.
Widget _buildDailyLogSection() {
  if (_selectedDay == null || !_dailyData.containsKey(_selectedDay))
    return SizedBox();
  final data = _dailyData[_selectedDay]!;
  return Card(
    elevation: 4,
    child: Padding(
      padding: EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text("Daily Log - ${_selectedDay!.toString().substring(0, 10)}",
              style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
          SizedBox(height: 10),
          if (data['symptoms'] != null)
            _buildLogItem("Symptoms",  data['symptoms'].join(',  ')),
          if (data['flow'] != null) _buildLogItem("Flow", data['flow']),
          if (data['notes'] != null) _buildLogItem("Notes", data['notes']),
Widget _buildLogItem(String title, String value) {
  return Padding(
    padding: EdgeInsets.symmetric(vertical: 4),
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text("$title: ", style: TextStyle(fontWeight: FontWeight.bold)),
        Expanded(child: Text(value))
// Build a card listing upcoming period start dates within the next 5 months.
Widget _buildUpcomingPeriods() {
```

```
 final upcoming = _predictedPeriodStarts
        .where((d) =>
        d.isAfter(DateTime.now()) &&
        d.isBefore(DateTime.now().add(Duration(days:  150))))
    .toList();
 if (upcoming.isEmpty) return SizedBox();
 return Card(
  elevation: 4,
  margin: EdgeInsets.symmetric(vertical: 10),
  child: Padding(
   padding: EdgeInsets.all(16),
   child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
     Text("upcoming Period Dates (Next 5 months)",
       style: TextStyle(
         fontSize: 16,
         fontWeight: FontWeight.bold,
         color: Colors.deepPurple)),
     SizedBox(height: 10),
     Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: upcoming
        .map((date) => Padding(
          padding: const EdgeInsets.symmetric(vertical: 2.0),
          child: Text(date.toString().substring(0, 10),
            style: TextStyle(fontSize: 14)),))
        .toList(),
// Build the small confirmation box that appears on the predicted period start date.
Widget _buildPeriodConfirmationBox() {
 // Check if there is an upcoming predicted period date that is today.
 if (_predictedPeriodStarts.isNotEmpty) {
```

```
DateTime today = DateTime.now();

            DateTime nextPredicted = _predictedPeriodStarts.firstWhere(
   (d) => !d.isBefore(today),
   orElse: () => _predictedPeriodStarts.last);
if (isSameDate(nextPredicted, today)) {
return Card(
   color: Colors.orange.shade50,
   margin: EdgeInsets.symmetric(vertical: 10),
   child: Padding(
    padding: EdgeInsets.all(16),
    child: Column(
     crossAxisAlignment: CrossAxisAlignment.start,
     children: [
      Text("Did your period start today (${today.toString().substring(0, 10)})?",
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          color: Colors.deepOrange)),
      SizedBox(height: 10),
      Row(
       mainAxisAlignment: MainAxisAlignment.end,
       children: [
        ElevatedButton(
         onPressed: () {
          // If yes, update the last period start to today.
          setState(() {
           _lastPeriodStart = today;
           _calculatePredictions(today); });},
        child: Text("Yes"),
        style: ElevatedButton.styleFrom(iconColor: Colors.green),),
       SizedBox(width: 10),
       ElevatedButton(
```

```
                onPressed: () async {
                        // If no, ask the user to pick a new period start
                  date. DateTime? newStart = await
                  showDatePicker( context: context,
                    initialDate: today,
                    firstDate: today.subtract(Duration(days: 30)),
                    lastDate: today.add(Duration(days: 30)),);
                  if (newStart != null) {
                    setState(() {
                      _lastPeriodStart = newStart;
                      _calculatePredictions(newStart); });}},
                child: Text("No"),
                style: ElevatedButton.styleFrom(iconColor: Colors.red),
  return SizedBox();}
// Get color based on the logged flow level.
Color _getFlowColor(String flow) {
switch (flow) {
    case 'Light':
      return Colors.blue.shade200;
    case 'Medium':
      return Colors.blue.shade400;
    case 'Heavy':
      return Colors.blue.shade800;
    default:
      return Colors.transparent; }}
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Women's Health Tracking'),
      actions: [
        IconButton(
```

```
            icon: Icon(Icons.settings),

                    onPressed:
      _showCycleSettings,),],), body:
      SingleChildScrollView(
        child: Padding(
          padding: EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              _buildCycleInfoCard(),
              SizedBox(height: 20),
              _buildCalendar(),
              SizedBox(height: 20),
              _buildPhaseLegend(),
              SizedBox(height: 20),
              _buildDailyLogSection(),
              _buildUpcomingPeriods(),
              _buildPeriodConfirmationBox(),],),)}
class DailyLogDialog extends StatefulWidget {
final Map<String, dynamic>? initialData;
DailyLogDialog({this.initialData});
  @override
  _DailyLogDialogState createState() => _DailyLogDialogState();}
class _DailyLogDialogState extends State<DailyLogDialog> {
final _formKey = GlobalKey<FormState>();
  final List<String> _selectedSymptoms = [];
  String? _flowLevel;
  final TextEditingController _notesController = TextEditingController();
  final List<String> _symptomsList = [
    'Cramps',
    'Headache',
    'Bloating',
```

```
  'Fatigue',
  'Mood swings'];
      @override
void initState() {
super.initState();
  if (widget.initialData != null) {
    _selectedSymptoms.addAll(widget.initialData!['symptoms'] ?? []);
    _flowLevel = widget.initialData!['flow'];
    _notesController.text = widget.initialData!['notes'] ?? ''; }}
@override
Widget build(BuildContext context) {
  return AlertDialog(
    title: Text("Daily Log"),
    content: Form(
      key: _formKey,
      child: SingleChildScrollView(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text("Symptoms:", style: TextStyle(fontWeight: FontWeight.bold)),
            Wrap(
              children: _symptomsList
                  .map((symptom) => FilterChip(
                        selected: _selectedSymptoms.contains(symptom),
                        label: Text(symptom),
                        onSelected: (selected) => setState(() {
                          if (selected) {
                            _selectedSymptoms.add(symptom);
                          } else {
                            _selectedSymptoms.remove(symptom)  }
                  .toList(),),
            SizedBox(height: 16),
```

```
            DropdownButtonFormField<String>(
              value: _flowLevel,
                decoration: InputDecoration(
                labelText: 'Flow Level',
                border: OutlineInputBorder(),),
              items: ['Light', 'Medium', 'Heavy']
                  .map((level) => DropdownMenuItem(
                        value: level,
                        child: Text(level),))
                  .toList(),
              onChanged: (value) => _flowLevel = value,),
            SizedBox(height: 16),
            TextFormField(
              controller:  _notesController,
              decoration:  InputDecoration(
              labelText: 'Notes',
                border:  OutlineInputBorder(),),
              maxLines: 3
      actions:       [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: Text("Cancel"), ),
        ElevatedButton(
          onPressed: () {
          Navigator.pop(context, {
            'symptoms': _selectedSymptoms,
            'flow': _flowLevel,
            'notes': _notesController.text,} },
          child: Text("Save"),

final TextEditingController _symptomController = TextEditingController();
List<Map<String, dynamic>> _diagnosisHistory = [];
bool _showWebView = false;
```

```
late InAppWebViewController _webViewController;
double _progress = 0;
      final double _appBarExtensionHeight = 70.0;
@override
void dispose() {
 _symptomController.dispose();
 super.dispose();}
@override
Widget build(BuildContext context) {
 return Scaffold(
   appBar: _showWebView
     ? _buildExtendedAppBar(context)
     : AppBar(
       title: Text("Symptom Checker"),
       actions: [
         IconButton(
           icon: Icon(Icons.history),
           onPressed: _showHistoryDialog,  ),],),
   body: _showWebView ? _buildWebMDView() : _buildMainContent(),);}
 PreferredSizeWidget _buildExtendedAppBar(BuildContext context) {
 return AppBar(
   title: Text("
   Input your
symptoms"),
   leading: IconButton(
     icon: Icon(Icons.arrow_back),
     onPressed: () => setState(() => _showWebView = false),),
   toolbarHeight: kToolbarHeight + _appBarExtensionHeight,
   flexibleSpace: Column(
   mainAxisAlignment: MainAxisAlignment.end

     children: [
     Container(
```

```
          height: kToolbarHeight,
          child: Row(
           children: [
            Expanded(
            child: Align(
              alignment: Alignment.centerLeft,
              child: Text(
               "              Symptom Checker",
               style: TextStyle(fontSize: 20)
        Container(
          height: _appBarExtensionHeight,
          color: Theme.of(context).appBarTheme.backgroundColor,
Widget _buildMainContent() {
  return SingleChildScrollView(
    padding:  EdgeInsets.all(16),
    child: Column(
     children: [
      Card(
      elevation: 4,
       child: Padding(
        padding: EdgeInsets.all(16),
        child: Column(
         children: [
          Text(
           "Check Your Symptoms",
           style: TextStyle(
           fontSize: 20,
            fontWeight: FontWeight.bold),),
          SizedBox(height: 20),
          ElevatedButton(
```

```
            onPressed: () => setState(() => _showWebView = true),
            child: Text("Use WebMD Symptom Checker"),
            style: ElevatedButton.styleFrom(
              padding: EdgeInsets.symmetric(vertical: 15),),),
          SizedBox(height: 20),
          Text(
            "Or describe your symptoms below:",
            style: TextStyle(fontSize: 16),),
          SizedBox(height: 10),
          TextField(
            controller: _symptomController,
            decoration: InputDecoration(
              hintText: "e.g. headache, fever, nausea...",
              border: OutlineInputBorder(),),
            maxLines: 3,),
          SizedBox(height: 15),
          ElevatedButton(
            onPressed: _addToHistory,
            child: Text("Analyze Symptoms"),),],),),),
      SizedBox(height: 20),
      _buildHistoryPreview();
Widget _buildWebMDView() {
return Stack(
  children: [
    Transform.translate(
      offset: Offset(0, -_appBarExtensionHeight),
      child: Container(
        height: MediaQuery.of(context).size.height + _appBarExtensionHeight,
        child: InAppWebView(
          initialUrlRequest: URLRequest(url: WebUri("https://symptoms.webmd.com/")),
          initialOptions: InAppWebViewGroupOptions(
            crossPlatform: InAppWebViewOptions(
```

```
            transparentBackground: true,
            disableVerticalScroll: true,
            disableHorizontalScroll: true,
            javaScriptEnabled: true,
          ),
        ),
        onWebViewCreated: (controller) {
          _webViewController = controller;
        },
    // Progress bar and bottom mask (keep existing)
    Positioned(
      top: 0,
      left: 0,
      right: 0,

      child: LinearProgressIndicator(
        value: _progress,
        backgroundColor: Colors.grey[200],),),
    Positioned(
      bottom: 0,
      left: 0,
      right: 0,
      child: Container(
        height: 80,
        color: Theme.of(context).scaffoldBackgroundColor,),),],); }
 void _addToHistory() {
  if (_symptomController.text.isEmpty) return;
  final mockDiagnosis = {
    'conditions': ["Common Cold"],
    'confidence': "Medium",
    'recommendation': "Rest and drink fluids",};
setState(() {
    _diagnosisHistory.insert(0, { mptoms':
```

```
        symptomController.text, 'diagnosis':
        mockDiagnosis,
        'date': DateTime.now().toString().substring(0, 10), });
      _symptomController.clear();}); }
  Widget _buildHistoryPreview() {
    if (_diagnosisHistory.isEmpty) return SizedBox();
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
          child: ListTile(
            title: Text(entry['diagnosis']['conditions'].join(', ')),
            subtitle: Text(entry['date'].substring(0, 10)),
            onTap: () => _showDiagnosisDetails(context, entry) ), )),],);}
  void _showHistoryDialog() {
    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: Text("Diagnosis History"),
        content: SizedBox(
          width: double.maxFinite,
          child: ListView.builder(
          shrinkWrap: true,
            itemCount: _diagnosisHistory.length,
            itemBuilder: (context, index) => ListTile(
              title: Text(_diagnosisHistory[index]['diagnosis']['conditions'].join(', ')),
              subtitle: Text(_diagnosisHistory[index]['date'].substring(0, 10)),
              onTap: () => _showDiagnosisDetails(context, _diagnosisHistory[index]), ),),),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: Text("Close"),
void _showDiagnosisDetails(BuildContext context, Map<String, dynamic> entry) {
```

```
    showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title:  Text("Diagnosis  Details"),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text("Symptoms: ${entry['symptoms']}"),
// Disease Diagnosis Page
class DiseaseDiagnosisPage extends StatefulWidget {
  @override
  _DiseaseDiagnosisPageState createState() => _DiseaseDiagnosisPageState();}
class _DiseaseDiagnosisPageState extends State<DiseaseDiagnosisPage> {
            SizedBox(height: 10),
            Text("Conditions: ${entry['diagnosis']['conditions'].join(', ')}"),
            SizedBox(height: 10),
actions: [
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: Text("Close"),),],),);}}
// Mood Detection Page (unchanged)
class MoodDetectionPage extends StatefulWidget {
  @override
  _MoodDetectionPageState createState() => _MoodDetectionPageState();}
class _MoodDetectionPageState extends State<MoodDetectionPage> {
final List<String> _moodOptions = [
    'Happy',
    'Sad',
    'Angry',
```

```
  – Tired',
  'h
  'Anxious',
  Thoughtful',
  'Excited',];
final List<String> _activities = [
  'Work',
  'Exercise',
  'Socializing',
  'Relaxing',
  'Studying',
  'Traveling',
  'Eating',
  'Sleeping',];
 String? _selectedMood;
 final List<String> _selectedActivities = [];
 final TextEditingController _notesController = TextEditingController();
 final Map<DateTime, Map<String, dynamic>> _moodEntries = {};

 void _logMood() async {
  if (_selectedMood == null) {
   ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text("Please select a mood before saving.")),);
   return; }
  final DateTime now = DateTime.now();
  setState(() {
   _moodEntries[now] = {
    'mood': _selectedMood,
    'activities': List.from(_selectedActivities),
    'notes': _notesController.text, };});
  // Clear the form after logging
  _selectedMood = null;
```

```
  _selectedActivities.clear();
  _notesController.clear();
ScaffoldMessenger.of(context).showSnackBar(
SnackBar(content: Text("Mood logged successfully!")),); }
Widget _buildMoodSelector() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text("How are you feeling today?",
          style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
      SizedBox(height: 10),
      Wrap(
        spacing: 10,
        runSpacing: 10,
        children: _moodOptions
                .map((mood) =>
                ChoiceChip( label:
                Text(mood),
                selected: _selectedMood == mood,
                onSelected: (selected) {
                setState(() {
                    _selectedMood = selected ? mood : null; });},))
            .toList(),),],);}
Widget _buildActivitySelector() {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text("What activities did you do today?",
          style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
      SizedBox(height: 10),
      Wrap(
        spacing: 10,
```

```
        runSpacing: 10,
        children: _activities
            .map((activity) => FilterChip(
                label: Text(activity),
                selected: _selectedActivities.contains(activity),
                onSelected: (selected) {
                  setState(() {
                    if (selected) {
                      _selectedActivities.add(activity);
                    } else {
                      _selectedActivities.remove(activity);  }});},))
            .toList(),),],);}
  Widget _buildNotesField() {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text("Notes (optional):",
          style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
        SizedBox(height: 10),
        TextField(
          controller: _notesController,
          decoration: InputDecoration(
          hintText: "Write about your day...",
          border: OutlineInputBorder(),),
          maxLines: 3,),],); }
  Widget _buildMoodLogs() {
    if (_moodEntries.isEmpty) {
    return Center(
        child: Text("No mood entries yet. Start logging your mood!",
            style: TextStyle(fontSize: 16, color: Colors.grey)),); }
    return ListView.builder(
      shrinkWrap: true,
```

```
    physics: NeverScrollableScrollPhysics(),
    itemCount: _moodEntries.length,
    itemBuilder: (context, index) {
      final entry = _moodEntries.entries.toList()[index];
      final date = entry.key;
      final data = entry.value;
      return Card(
        margin: EdgeInsets.symmetric(vertical: 5),
        child: ListTile(
          title: Text(data['mood'] ?? ''),
          subtitle: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              if (data['activities'] != null && data['activities'].isNotEmpty)
                Text("Activities: ${data['activities'].join(', ')}"),
              if (data['notes'] != null && data['notes'].isNotEmpty)

                  Text("Notes: ${data['notes']}"),],),
          trailing: Text(
          "${date.day}/${date.month}/${date.year}",
          style: TextStyle(color: Colors.grey
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("Mood Journal")),
    body: SingleChildScrollView(
      padding: EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          _buildMoodSelector(),
          SizedBox(height: 20),
          _buildActivitySelector(),
```

```
            SizedBox(height: 20),
            _buildNotesField(),
            SizedBox(height: 20),
            ElevatedButton(
            onPressed: _logMood,
            child: Text("Log Mood"),
              style: ElevatedButton.styleFrom(
                padding: EdgeInsets.symmetric(vertical: 15),
                backgroundColor: Colors.deepPurple,
                foregroundColor: Colors.white,), ),
            SizedBox(height: 20),
            Text("Mood History",
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
            SizedBox(height: 10),
            _buildMoodLogs(),],),),);}}
```

ith Dialogflow (unchanged)

```
 class ChatBotPage extends StatefulWidget {
 @override
 _ChatBotPageState createState() => _ChatBotPageState();
}

class _ChatBotPageState extends State<ChatBotPage> {
 late DialogFlowtter dialogFlowtter;
 final TextEditingController _controller = TextEditingController();
 List<Map<String, dynamic>> messages = [];
@override
 void initState() {
  super.initState();
  DialogFlowtter.fromFile().then((instance) => dialogFlowtter = instance); }
 void sendMessage(String text) async {
  if (text.isEmpty) return;
```

```
    setState(() {
      messages.add({"message": text, "isUser": true}); });
    DetectIntentResponse response = await dialogFlowtter.detectIntent(
      queryInput: QueryInput(text: TextInput(text: text)), );
    if (response.message != null) {
      setState(() {
        messages.add({
          "message": response.message?.text?.text?[0],
          "isUser": false});});}
    _controller.clear();}
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("AI Medical Chatbot")),
      body: Column(
        children: [
          Expanded(
            child: ListView.builder(
            itemCount: messages.length,
             itemBuilder: (context, index) => ListTile(
               title: Text(messages[index]["message"],
                 textAlign: messages[index]["isUser"]
                   ? TextAlign.right
                   : TextAlign.left),),),),
          ChatInputWidget(sendMessage: sendMessage, controller: _controller),], ),); }}
// Chat Input Widget (unchanged)
class ChatInputWidget extends StatelessWidget {
  final Function(String) sendMessage;
  final TextEditingController controller;
  ChatInputWidget({required this.sendMessage, required this.controller});
  @override
  Widget build(BuildContext context) {
```

```
    return Padding(
      padding: EdgeInsets.all(8.0),
      child: Row(
        children: [
          Expanded(
            child: TextField(
              controller: controller,
              decoration: InputDecoration(hintText: "Type a message..."),
              onSubmitted: (text) {
                if (text.isNotEmpty) {
                  sendMessage(text);
                  controller.clear();}},),),
          IconButton(
            icon: Icon(Icons.send, color: Colors.deepPurple),
            onPressed: () {
              if (controller.text.isNotEmpty) {
                sendMessage(controller.text)
```

## 6.2 Message.dart

```
import 'package:flutter/material.dart';
class MessagesScreen extends StatefulWidget {
  final List messages;
  const MessagesScreen({Key? key, required this.messages}) : super(key: key);
  @override
  _MessagesScreenState createState() => _MessagesScreenState();}
class _MessagesScreenState extends State<MessagesScreen> {
@override
  Widget build(BuildContext context) {
    var w = MediaQuery.of(context).size.width;
    return ListView.separated(
        itemBuilder: (context, index) {
          return Container(
```

```
        margin: EdgeInsets.all(10),
        child: Row(
         mainAxisAlignment: widget.messages[index]['isUserMessage']
            ? MainAxisAlignment.end
            : MainAxisAlignment.start,
         children: [
          Container(
            padding: EdgeInsets.symmetric(vertical: 14, horizontal: 14),
            decoration: BoxDecoration(
              borderRadius: BorderRadius.only(
              bottomLeft: Radius.circular(20,),
              topRight: Radius.circular(20),
              bottomRight: Radius.circular(
                  widget.messages[index]['isUserMessage'] ? 0 : 20),
               topLeft: Radius.circular(
                  widget.messages[index]['isUserMessage'] ? 20 : 0),),
              color: widget.messages[index]['isUserMessage']
                ? Colors.grey.shade800
                    : Colors.grey.shade900.withOpacity(0.8)),
            constraints: BoxConstraints(maxWidth: w * 2 / 3),
            child:
              Text(widget.messages[index]['message'].text.text[0])),],),); },
      separatorBuilder: (_, i) => Padding(padding: EdgeInsets.only(top: 10)),
      itemCount: widget.messages.length); }
}
```

## 6.3 Backend

```
{
  "type": "service_account",
  "project_id":  "bensonflutterboard-knlg",
```

"private_key_id": "7518ea8bc3f1ce1456c8dc651216620245f156ee",

"private_key": "-----BEGIN PRIVATE KEY-----
\nMIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQCo3MzmQrAvY
ZIP\nfceundE03h0F+eEK/HRPNa/mQWyd5syTWDKVhoiI9KmlPOAPxMLfZ5nLqb+3pqh
v\n7Q1ndeAeHsmbCji9E2XJfP+FgwN06PWW12QCw0oD7cLsxhXVy6ZSsiUG63InWAmZ
\nh9NEczHemwGhngDz9pr6slbgro5PXiouAhNsrhFcG9BZ8LE9DY6VH1nqhTYfAEGt\naj
Y62wbwTfevpNGxckTgIEZnhg/d5GGbh/ssE8xXQKOToo4OZv+z8cPqHDpL0g0S\nT0tX5
ELIBJjspLSm4u5egI7MvzpgCp/6RQ3J9VEXbdIPvkpQuGjfc8VHpX+m3Jzs\n/koBBAULA
gMBAAECggEACo9MQktXUznfIBg13YXsUvRfjO50aUh3nuB08ImU2PGu\n8g6Tkzrc3P
ECjt0WB0ZpsqjQOvvm4P2LY6cdAyRMRfXs65ohtDPmAJ9QJ1xAYhY+\nDWxzdPdwxdA
fiXOrDTn1W7nc4Og7tO7SItzAlHOTtx+rDsf1DUyVon8hObRmHTEP\nNZN4/NEH4zdWk
FNb+FOxIf+W2KjxQHjtE5+njq1OLsVCnuVEq9VzKZwoDLc4/psg\nBqghNbQtKqqBO4m
D5YVb56oP4fox2Zxs72xvLNweVBi6MIg0h3fXkWBTDRAT4jWa\nRQzf4Ri8j+mwjjZ2gR
nMlZZrJYKeEuqtSKbsA0aecQKBgQDh4ZE5IRAGlg6+rVZ7\nm3jy6ll7pby6Gq14BLQpu
QJGYTNvBv5OtXcawhlr6vE7v26ZnJutFg90yOVkq8LD\nSJrHG4zTPh6J8oI7cMrsZVR3E
yvAbNCpCqAF7r0z9ladkZfyRi902S7Motv32FJM\nQ/zzsmQYM+zRofZT7dF6mU/cwwKB
gQC/YOhKhp/PQBz9ikBzUvAm9hSqmFUDmeog\nI03OJaMP96Qzp+MRFaAncsq4s43Jy
AsixSzmzRN3bDrP7Lh0PIF8rhejMVOr57v5\nHixHi+9lav1j7K3a3e4O5qkSgBh5WbFCPq
RqkqpjWaxphBXZzlUJJqCRiOC0K3Sm\nXEvwOmtSGQKBgF2IzFv9xucTSDPJD2DL9n+
Qb1F2RfpChcaHHBuS0tBV+7hkCvaI\nyjM4YNKBTTdSg/f0E7rOwcO4VNUlEtdOv1jnnZ
7zwC2bUDZQ9JaDN0C/nYRnCtiU\n38LVI1/bB+7jlSnrykb2kByI69FrsFiIZrx7CFvFVLFS

AfCrL+02gw2FAoGAZxHq\nCTk4GIaPFn2++F1SuakOuFISdz4NugFJhhz/FTpqVOe2gx0C
hDxuscsCMgpq9O78\nn9ETCt5fTlxPe0qa8mtQj7OhPJQxyJlXf7D93cEhdw+hEp8z9xKDP
FOA2tpTfTn/\nkuYqnhdNpAZfPk7BvYvwLnFpk59T08UhoN/VrQECgYEAzFgR6d6d1fP+
pBTrN/Wa\nq/XDTW9rETYM4gCl79UXsOPoXrJh6xd6tm8YVtpqJwBSsOMGsfjdhlCQ0o
w2Kp5T\nrGtwAKOC6IoGCJfS0X7L3QWRd3TC3TSr9ZCahUTftFXZJwC0zrWd770b0jX
EGuoi\nOVSKXzeuq1btHi8I58Ukf8Q=\n-----END PRIVATE KEY --- \n",

"client_email": "medbot@bensonflutterboard-knlg.iam.gserviceaccount.com",

"client_id": "117862856480622696689",

"auth_uri": "https://accounts.google.com/o/oauth2/auth",

"token_uri": "https://oauth2.googleapis.com/token",

"auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",

"client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/medbot%40bensonflutterboard-knlg.iam.gserviceaccount.com",

"universe_domain": "googleapis.com" }

# CHAPTER 7

# SYSTEM TESTING

## 7.1 Test Plan

The test plan outlines the key testing categories for the application, ensuring core functionalities and performance expectations are met. Functionality testing includes verifying user login/signup, checking the AI chatbot's accuracy, and assessing the correctness of period and ovulation tracking. Security testing ensures that all user data is encrypted and securely managed. Performance testing checks the app's ability to handle multiple users without lag, while usability testing evaluates the user interface for ease of navigation. All tests under the plan have been marked as verified.

| Test ID | Test Category | Description | Expected Result | Status |
|---|---|---|---|---|
| TP1 | Functionality Testing | Verify user login/signup | User's should be able to register and login successfully | Verified |
| TP2 | Functionality Testing | Check AI chat bot accuracy | AI should provide relevant medical advice | Verified |
| TP3 | Functionality Testing | Period and ovulation tracking accuracy | Correct predictions based on input data | Verified |
| TP4 | Security testing | Ensure data encryption for user data | User data should be securely stored and transmitted | Verified |

| | | | | |
|---|---|---|---|---|
| TP5 | Performance Testing | Load test for multiple concurrent users | App should handle multiple users without lag | Verified |
| TP6 | Usability Testing | UI/UX validation for ease of navigation | User's should navigate the app seamlessly | Verified |

**Fig. 7.1: Test Plan**

## 7.2 Test Case

The test cases specify objectives and expected outcomes for each functionality. TC1 verifies the accuracy of period cycle predictions, while TC2 focuses on mood detection from input text. TC3 checks if the chatbot provides accurate responses to medical queries. TC4 evaluates disease prediction based on symptoms. TC5 ensures that proactive alerts and notifications are triggered appropriately. TC6 confirms the implementation of secure data practices, including encryption and anonymization.

| Test Case ID | Objective | Expected Result |
|---|---|---|
| TC1 | Predict period cycle accuracy. | Predict next cycle start. |
| TC2 | Detect mood, based on input text. | Classify mood as happy, sad neutral etc. |
| TC3 | Chat bot responds correctly to medical queries. | Suggest possible conditions or advices. |

| | | |
|---|---|---|
| TC4 | Symptom based disease prediction. | Display probable diseases. |
| TC5 | Proactive alerts are triggered correctly. | Notifications sent when needed. |
| TC6 | Ensure data security and complaints. | Data is encrypted anonymized and secured. |

**Fig. 7.2: Test Case**

### 7.3 Traceability Matrix

The traceability matrix maps each requirement to its corresponding test case, ensuring that all functional aspects of the application are tested. R1 (Period and ovulation tracking) is linked to TC1, R2 (Mood journal using NLP) to TC2, and R3 (AI chatbot for medical help) to TC3. Similarly, R4 (Symptom-based disease prediction) corresponds to TC4, R5 (Proactive alerts and insights) to TC5, and R6 (Secure data management) to TC6. This matrix ensures all requirements are accounted for and validated through dedicated test cases.

| Requirement ID | Requirement Description | Associated test case |
|---|---|---|
| R1 | Period and ovulation tracking algorithm implementation. | TC1 |
| R2 | Mood Journal using NLP and psychological data. | TC2 |
| R3 | Ai chat bot for real time medical assistance. | TC3 |

| | | |
|---|---|---|
| | | |
| R4 | Symptom analysis and disease prediction. | TC4 |
| R5 | Proactive alerts and personalized insights | TC5 |
| R6 | Secure data management and privacy | TC6 |

**Fig. 7.3: Traceability Matrix**

# CHAPTER 8

# RESULT

## 8.1 OUTPUT

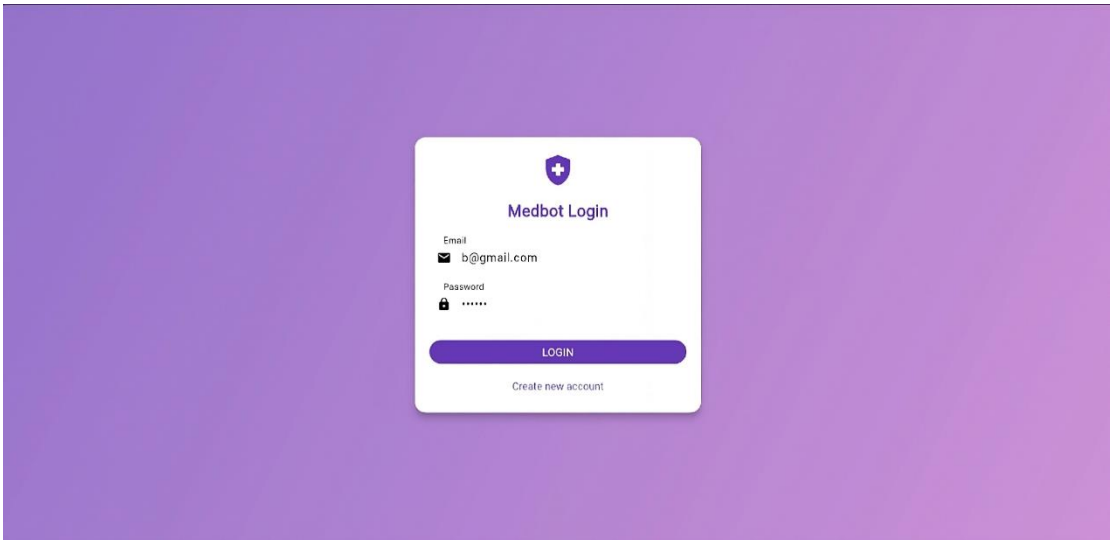


**Fig. 8.1: Login Page**


**Fig. 8.2: Home Page**

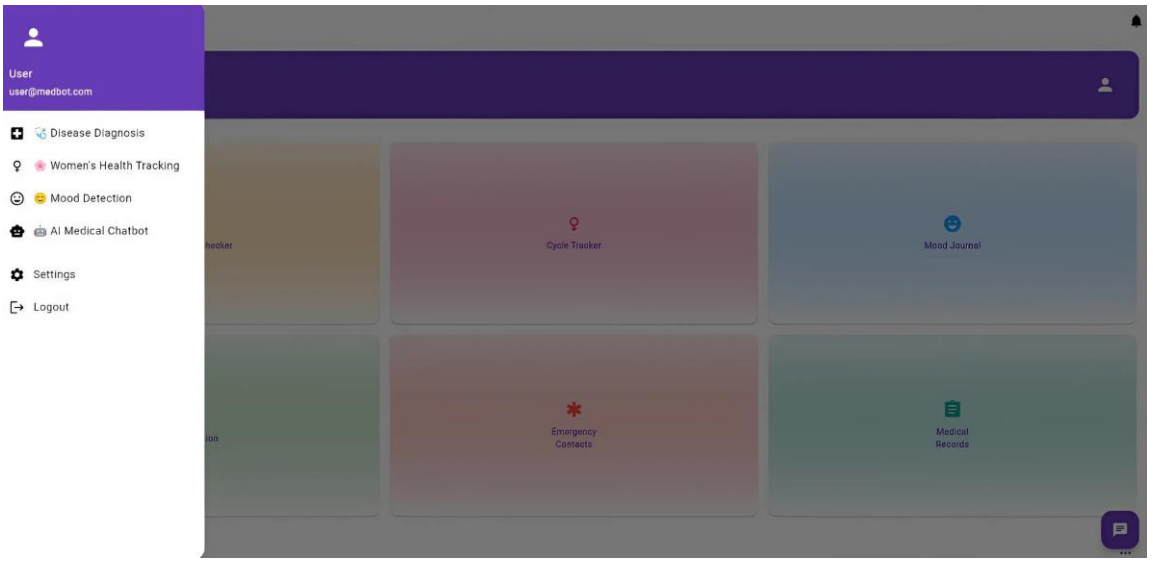


**Fig. 8.3: Side Bar**

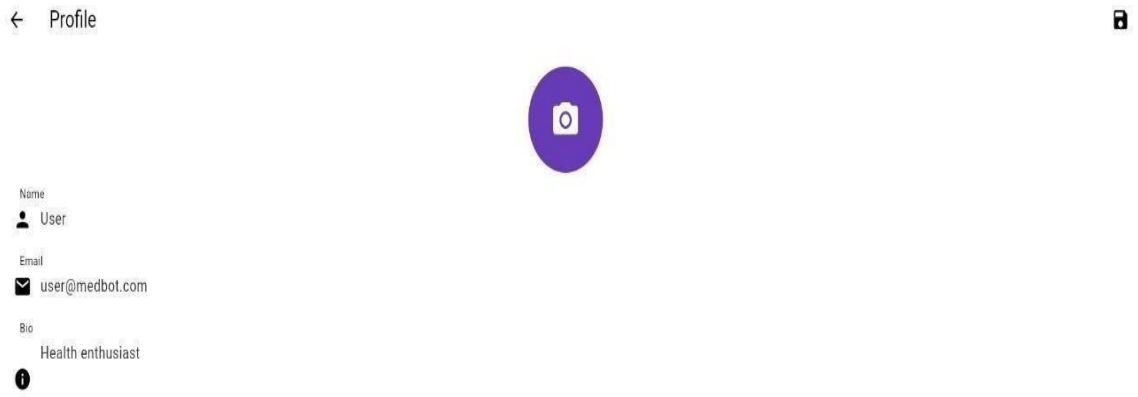


**Fig. 8.4: User Profile**


**Fig. 8.5: AI Chat Bot**

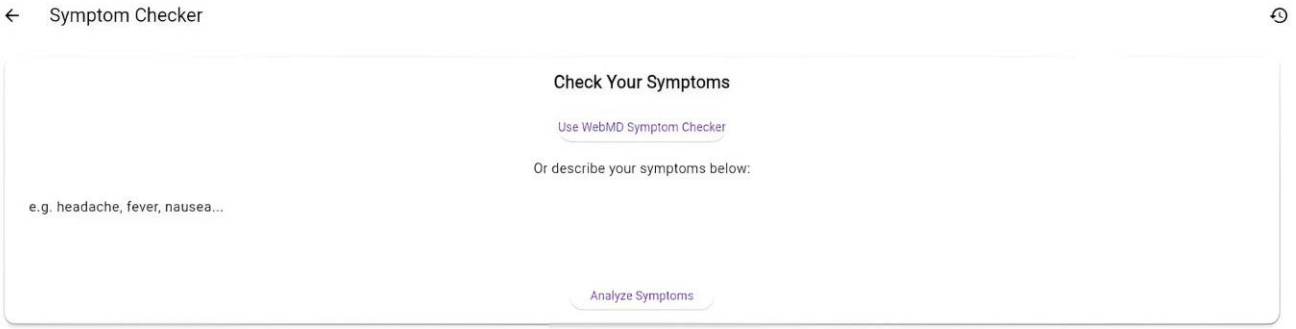


**Fig. 8.6: Symptom Checker**

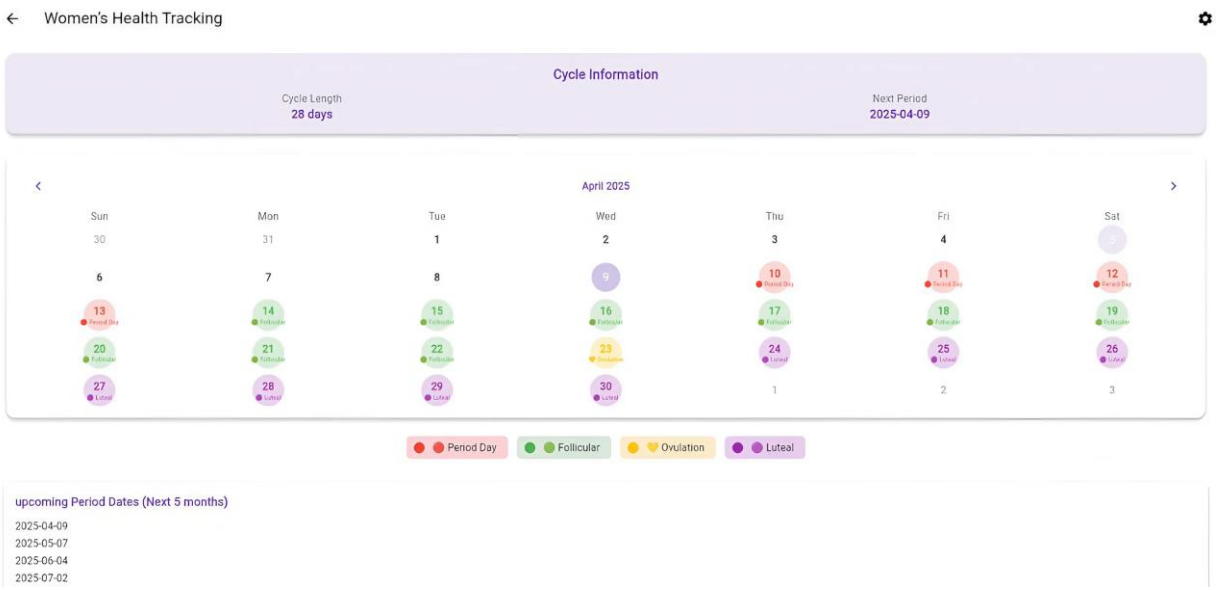


**Fig. 8.7: Women's Health Tracking**

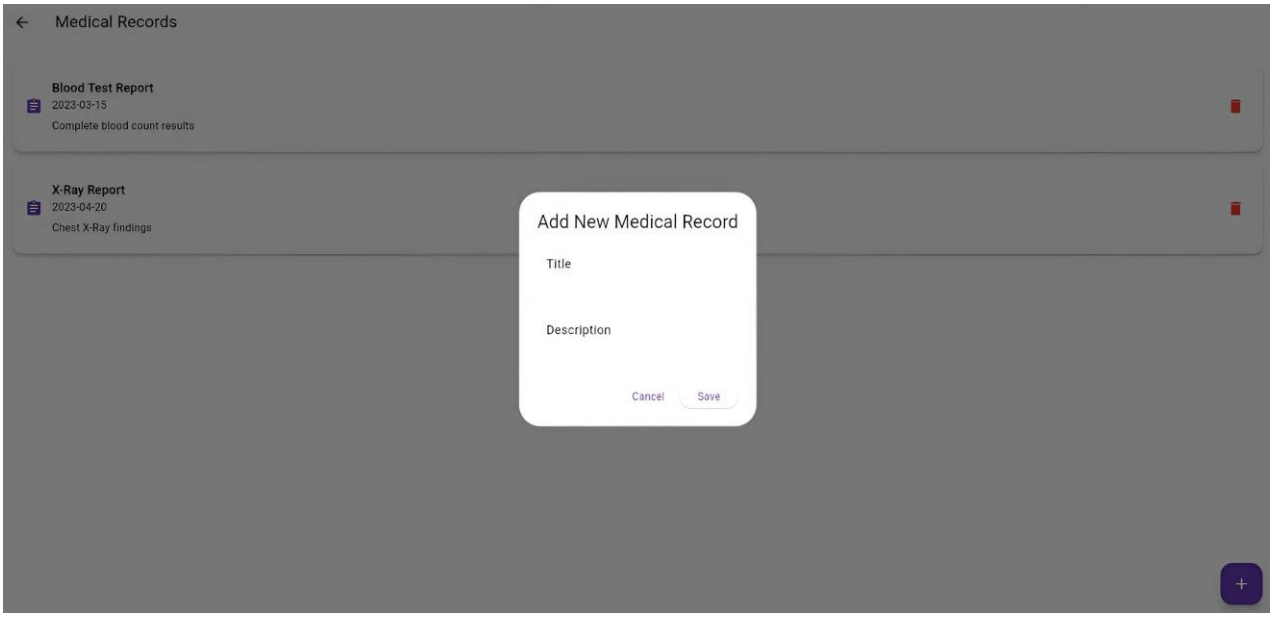


**Fig. 8.8: Medical Records**

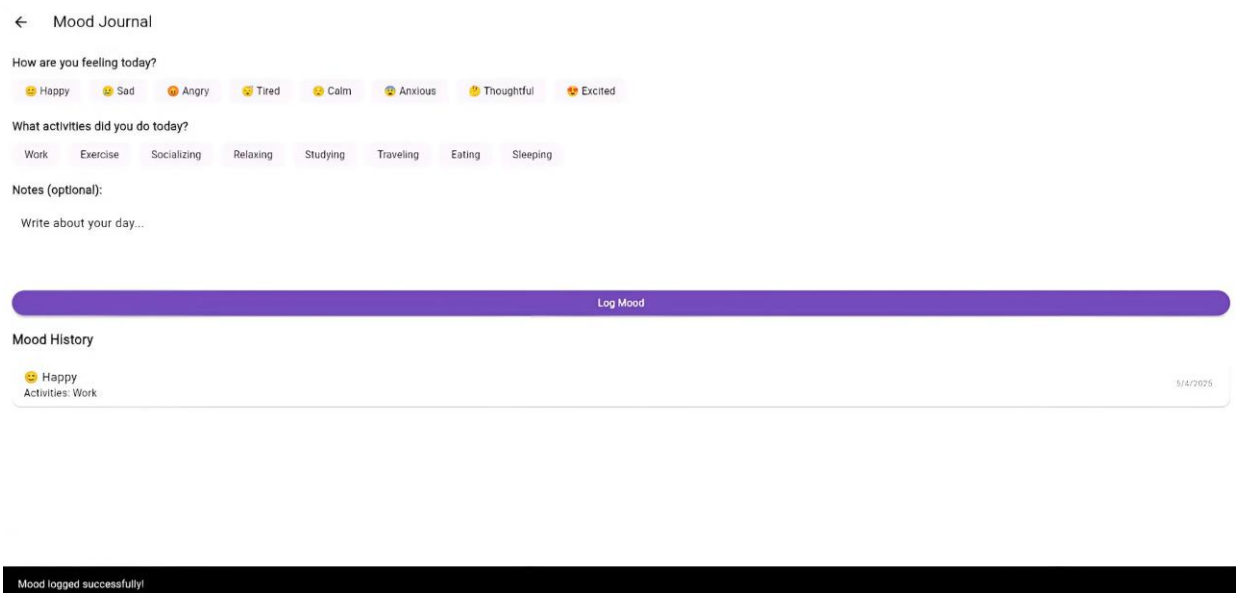


**Fig. 8.9: Mood Journal**

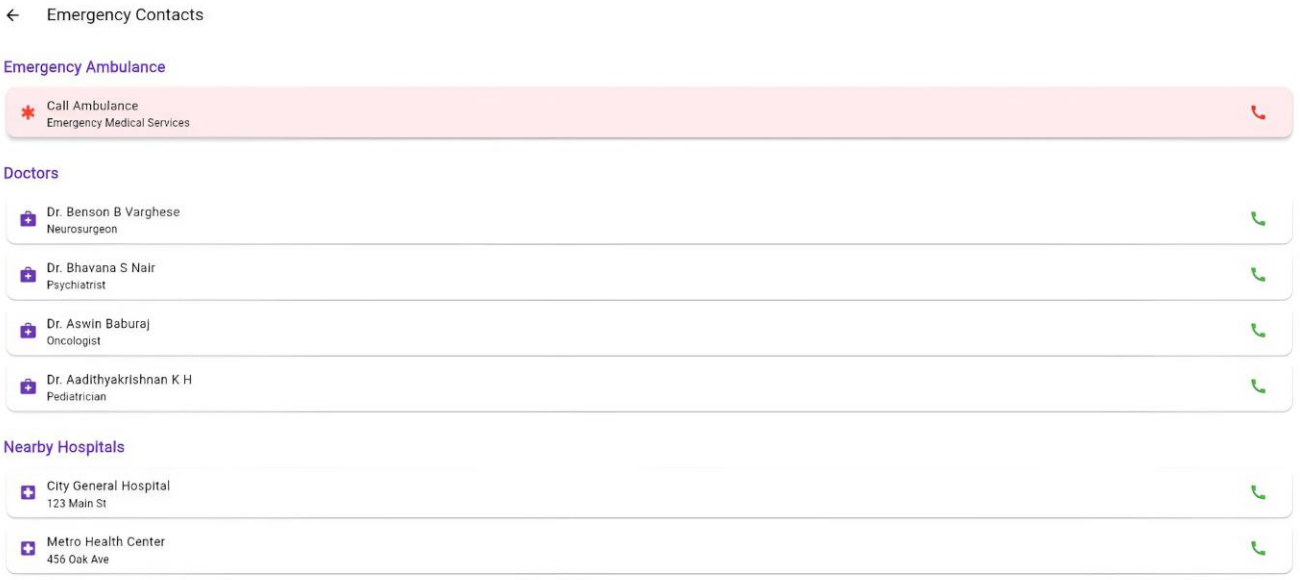


**Fig. 8.10: Emergency Contact**

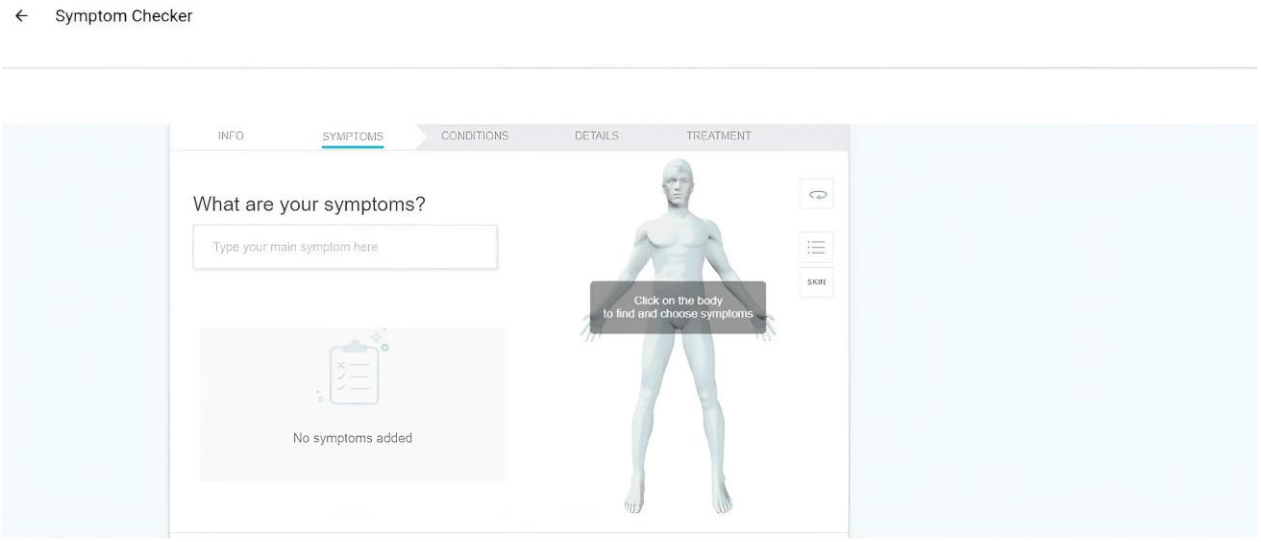


**Fig. 8.11: Symptom Checker with WebMD**

# CHAPTER 9

# CONCLUSION AND FUTURE SCOPE

## 9.1 Conclusion

The proposed AI-powered Disease Tracking and Prediction System overcomes the limitations of traditional disease monitoring methods by incorporating machine learning, real-time data processing, and predictive analytics. It offers key benefits such as faster outbreak detection, improved resource allocation, and proactive public health management. The system features include real-time disease monitoring, predictive outbreak forecasting, automated alerts for healthcare authorities, AI-powered health assistance for users, and personalized health insights. Designed for scalability and adaptability, it can integrate with wearable devices and diverse healthcare data sources, making it suitable for deployment at global, national, and regional levels, and adaptable to emerging diseases and health threats. It upholds high standards of security with HIPAA and GDPR compliance, ensures efficient performance, and offers a user-friendly experience. Ultimately, the system aims to enhance early disease detection and outbreak prevention, improve decision-making through data-driven insights, enable efficient and proactive interventions, and contribute to better health outcomes worldwide.

## 9.2 Future Scope

Looking ahead, the future scope includes advanced predictive modeling through the integration of deep learning, reinforcement learning, and neural networks, as well as the use of environmental, genetic, and socioeconomic data for comprehensive disease analysis. Genomic data integration will support the detection of new disease variants and improve outbreak predictions using genetic insights. AI chatbots powered by advanced natural language processing and sentiment analysis will enable better symptom analysis and user interaction. The system also plans to incorporate wider data sources, including social media and mobility trends, and expand data collection from wearables, IoT devices, and public health databases. A global health surveillance network will be established through connections with WHO, national health agencies, and real-time global monitoring systems, enabling quicker international responses to emerging outbreaks. Further developments will include long-term health predictions

based on lifestyle and historical data, seamless electronic health record (EHR) integration, and expansion into mental health tracking using biometric data and AI-driven insights. The system will also be tailored for accessibility in low-resource settings, offering cost-effective deployment solutions to ensure global reach and equity in healthcare.

# FUTURE SUGGESTION

The feature enhancements for the app focus on delivering a more personalized and intelligent user experience. Personalized insights and recommendations will offer users tailored health tips based on their cycle history, symptoms, and personal goals, supported by AI-driven suggestions for improving menstrual health and lifestyle. An advanced AI chatbot with natural language processing will be introduced to handle a broader range of queries, along with voice-based interaction for hands-free convenience. Predictive analytics powered by machine learning will enhance the accuracy of ovulation and fertility predictions, while also incorporating anomaly detection to alert users of irregular patterns. To elevate user experience, the app will feature an enhanced UI/UX with dark mode, accessibility options, customizable themes, intuitive dashboards, and data visualizations. Multilingual support will broaden accessibility, and an in-app user feedback system will foster continuous improvement based on user input.

Security and privacy will be strengthened through advanced data encryption, end-to-end protection, and biometric authentication such as fingerprint and facial recognition. The app will comply with global privacy regulations, including GDPR and HIPAA, ensuring data protection across regions. Performance improvements will include cloud integration for scalable, efficient data storage and reliable backups, along with regular load testing to maintain app responsiveness during high-traffic periods. Additional integrations will allow users to sync data from wearable health devices like Fitbit and Apple Watch for more accurate health predictions. Community and support features, such as user forums and support groups, will foster a sense of connection and shared experiences.

For monetization, the app will adopt a subscription model offering premium features like advanced tracking, detailed analytics, and AI-generated reports. In-app purchases will include curated health plans, personalized reports, and access to expert consultations, creating value-added services while supporting continued app development and innovation.

# REFERENCES

[1] Mana Saleh Al Reshan; Samina Amin; Muhammad Ali Zeb; Adel Sulaiman; Hani Alshahrani; Asadullah Shaikh, "A Robust Heart Disease Prediction System Using Hybrid Deep Neural Networks" , 2023, [Online], Available: https://ieeexplore.ieee.org/document/10302290

[2] Norma Latif Fitriyani; Muhammad Syafrudin; Ganjar Alfian; Jongtae Rhee, "Development of Disease Prediction Model Based on Ensemble Learning Approach for Diabetes and Hypertension", 2023, [Online], Available: https://ieeexplore.ieee.org/document/8854986

[3] Senthilkumar Mohan; Chandrasegar Thirumalai; Gautam Srivastava, "Effective Heart Disease Prediction Using Hybrid Machine Learning Techniques", 2022, [Online], Available: https://ieeexplore.ieee.org/document/8740989

[4] Yining Zhao; Yuelai Su, "Comparison of Three Prediction Models for the Incidence of Epidemic Diseases", 2023, [Online], Available: https://ieeexplore.ieee.org/document/9258817

[5] Haolin Wang; Zhilin Huang; Danfeng Zhang; Johan Arief; Tiewei Lyu; Jie Tian, "Integrating Co-Clustering and Interpretable Machine Learning for the Prediction of Intravenous Immunoglobulin Resistance in Kawasaki Disease", 2022, [Online], Available: https://ieeexplore.ieee.org/document/9097874

[6] Lei Wang, Hao Li, Yuqi Wang, Yihong Tan, Zhiping Chen, Tingrui Pei, Quan Zou, "MDADP: A Webserver Integrating Database and Prediction Tools for Microbe- Disease Associations", 2022, [Online], Available: https://pubmed.ncbi.nlm.nih.gov/35254998/

[7] Kai Zheng, Zhu-Hong You, Lei Wang, Yi-Ran Li, Ji-Ren Zhou, Hai-Tao Zeng, "MISSIM: An Incremental Learning-Based Model with Applications to the Prediction of miRNA-Disease Association", 2023, [Online], Available: https://pubmed.ncbi.nlm.nih.gov/32749964/

[8] Hamdi A. Al-Jamimi, "Synergistic Feature Engineering and Ensemble Learning for Early Chronic Disease Prediction", 2024, [Online], Available: https://ieeexplore.ieee.org/document/10511078